

Master Thesis

G-SVOM: Real time semantic voxel mapping

Autumn Term 2024

Declaration of Originality

I hereby submit the written work entitled:

G-SVOM: Real time semantic voxel mapping

- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it¹. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies².
- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it¹. Suggestions from the supervisor regarding language and content are excepted. I used and cited generative artificial intelligence technologies².
- ☒ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it¹. Suggestions from the supervisor regarding language and content are excepted. I used generative artificial intelligence technologies². In consultation with the supervisor, I did not cite them.

Author

Hynek

Zamazal

Student supervisors

Robin
Jonas

Schmid
Frey

Supervising lecturer

Marco

Hutter

With the signature, I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on The citation conventions usual to the discipline in question here have been respected. I have documented all methods, data, and processes truthfully and fully. Additionally, I have mentioned all persons who were significant facilitators of the work.

The above-written work may be tested electronically for plagiarism.

Zurich, 29. 10. 2024

Place and date



Signature

¹Co-authored work: The signatures of all authors are required. Each signature attests to the originality of the entire piece of written work in its final form.

²E.g. ChatGPT, DALL-E 2, Google Bard

Intellectual Property Agreement

The student acted under the supervision of Prof. Hutter and contributed to research of his group. Research results of students outside the scope of an employment contract with ETH Zurich belong to the students themselves. The results of the student within the present thesis shall be exploited by ETH Zurich, possibly together with results of other contributors in the same field. To facilitate and to enable a common exploitation of all combined research results, the student hereby assigns his rights to the research results to ETH Zurich. In exchange, the student shall be treated like an employee of ETH Zurich with respect to any income generated due to the research results.

This agreement regulates the rights to the created research results.

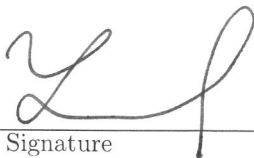
1. Intellectual Property Rights

1. The student assigns his/her rights to the research results, including inventions and works protected by copyright, but not including his moral rights ("Urheberpersönlichkeitsrechte"), to ETH Zurich. Herewith, he cedes, in particular, all rights for commercial exploitations of research results to ETH Zurich. He is doing this voluntarily and with full awareness, in order to facilitate the commercial exploitation of the created Research Results. The student's moral rights ("Urheberpersönlichkeitsrechte") shall not be affected by this assignment.
2. In exchange, the student will be compensated by ETH Zurich in the case of income through the commercial exploitation of research results. Compensation will be made as if the student was an employee of ETH Zurich and according to the guidelines "Richtlinien für die wirtschaftliche Verwertung von Forschungsergebnissen der ETH Zürich".
3. The student agrees to keep all research results confidential. This obligation to confidentiality shall persist until he or she is informed by ETH Zurich that the intellectual property rights to the research results have been protected through patent applications or other adequate measures or that no protection is sought, but not longer than 12 months after the collaborator has signed this agreement.
4. If a patent application is filed for an invention based on the research results, the student will duly provide all necessary signatures. He/she also agrees to be available whenever his aid is necessary in the course of the patent application process, e.g. to respond to questions of patent examiners or the like.

2. Settlement of Disagreements

Should disagreements arise out between the parties, the parties will make an effort to settle them between them in good faith. In case of failure of these agreements, Swiss Law shall be applied and the Courts of Zurich shall have exclusive jurisdiction.

Zurich, 29. 10. 2024
Place and date


Signature

Contents

Acknowledgement	v
Abstract	vii
Symbols	ix
1 Introduction	1
1.1 Problem statement	1
1.2 Related work	3
1.3 Contribution	5
2 Method	7
2.1 G-VOM	7
2.1.1 G-VOM explained	7
2.1.2 G-VOM modifications	9
2.2 Semantic label association	9
2.2.1 Benchmarks	10
2.2.2 Learned methods using density along the ray	12
2.2.3 Learned method using geometric context	16
2.3 G-SVOM architecture	17
3 Implementation details	19
3.1 Training and evaluation data	20
3.2 Training process and parameters	21
3.2.1 Benchmark thresholds	22
4 Experiments and Results	25
4.1 Performance on simulated data	25
4.1.1 Benchmark analysis	25
4.1.2 Single view accuracy	26
4.1.3 First voxel accuracy	28
4.1.4 Accuracy on labeled maps	29
4.1.5 Not seen semantic labels	32
4.2 Resource usage	33
4.2.1 Processing time	33
4.2.2 Memory requirements	34
4.3 Running G-SVOM on real data	36
4.4 Dependency on camera calibration	44
5 Conclusion	45
5.1 Findings	45
5.2 Limitations	46
5.3 Possible future work	46

Acknowledgement

I would like to thank my supervisors, Jonas Frey and Robin Schmid, for always giving me good advice, pushing me to do better, and supporting me when I struggled. I would not be able to finish the thesis without them.
I would also like to extend my gratitude to my friends and family for supporting me throughout my studies.

Abstract

For navigation in unstructured outdoor environments, it is vital to have an accurate representation of the full 3D geometry of the space around the robot. However, geometric information alone is often not enough to allow for the planning of safe and efficient paths. Including semantic information in the map is essential for detecting certain environmental properties crucial for robust navigation. Additionally, having semantic information in a map makes it more useful for other tasks than just path planning. In this thesis, we present a pipeline for building voxel maps from point clouds and merging semantic labels from images into the occupied voxels. For the semantic label to voxel association task, we show the limitations of standard methods based on projection and introduce a novel machine learning-based method to alleviate them. Unlike the state of the art, our novel method is agnostic to the intrinsic camera calibration parameters. We trained and evaluated our methods on a synthetic dataset, showing a 10.2 % labeling accuracy improvement over the best-performing projection-based benchmark. We also compared the performance of each method on a real-world dataset. The semantic voxel mapping pipeline is capable of running in real time because of parallelization on the GPU.

Symbols

Symbols

s	voxel edge length
l	voxel map side length (in voxels)
$\mathbf{i}$	3D index $(i_x \ i_y \ i_z)$ of a voxel
$\mathbf{o}_g$	voxel map origin (as a world coordinate frame aligned voxel grid index)
$\mathbf{x}$	robot position
n_p	number of point cloud points inside a voxel
n_r	number of rays passing through a voxel
d	density of a voxel
N	fixed length of a ray projected from a pixel
M	number of possible semantic labels
K	edge length of a geometric context cube
G	dimension of geometric context feature vector
$\mathbf{s} \in \mathbb{R}^M$	one-hot encoded semantic label
$\mathbf{d} \in \mathbb{R}^N$	a vector of voxel densities $(d_1 \ d_2 \ \dots \ d_N)$ along a ray
$\mathbf{y} \in \mathbb{R}^N$	a vector of label placements scores for each voxel along a ray

Indices

x	x axis
y	y axis
z	z axis
w	world coordinate frame
g	world coordinate frame aligned voxel grid
c	camera coordinate frame
$\mathbf{i}$	value for voxel with index $(i_x \ i_y \ i_z)$

Acronyms and Abbreviations

GPU	Graphics Processing Unit
SLAM	Simultaneous Localization and Mapping
RAM	Random Access Memory
VRAM	Video Random Access Memory

Chapter 1

Introduction

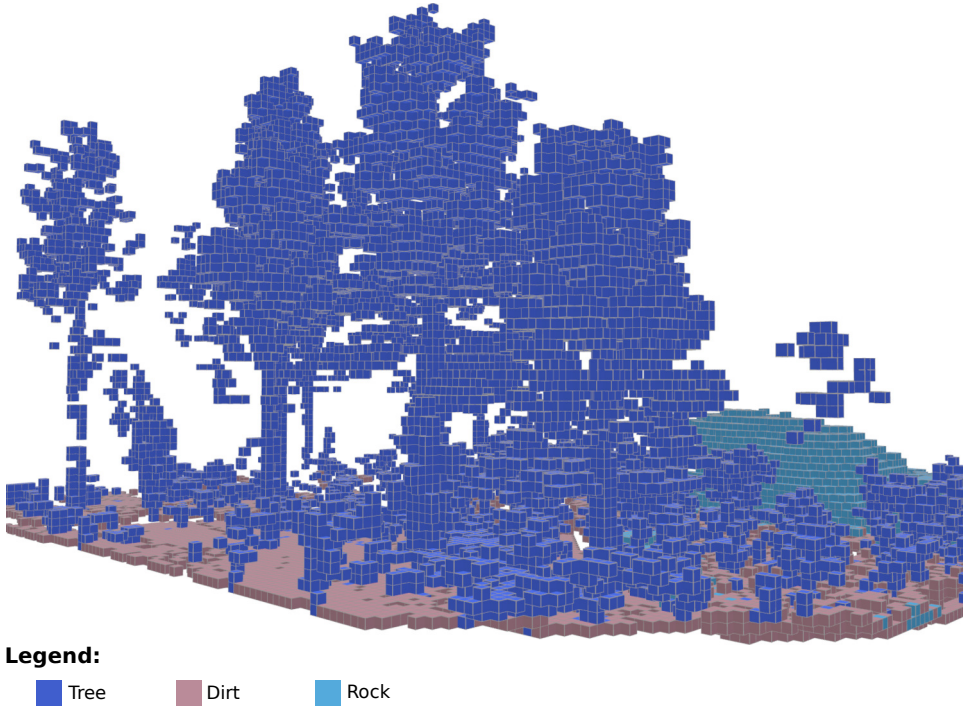


Figure 1.1: G-SVOM is a voxel mapping pipeline that uses novel machine learning-based methods to fuse semantic labels from images into the occupied voxels.

1.1 Problem statement

We want robots to operate outdoors - in unknown and unstructured environments with a wide range of terrains and an uncountable number of obstacle type and shape combinations. We need to understand the nature of these environments for the robot to be able to traverse them.

Robots are equipped with a wide array of sensors, capturing different aspects of the space around them. We need to process these measurements and decide what action the robot should take. There are two approaches to this: either use an end-to-end method, directly telling us the appropriate action from the raw sensor outputs or

first build a map of the environment and then use a separate planner to decide what to do. While end-to-end methods have become more and more popular in recent years because of their efficiency, the traditional map-and-planner approach is more versatile as, in addition to planning, maps can be used for higher-level tasks or passed to other robots to help them traverse already explored sections of the environment. Moreover, a well-defined mapping pipeline can serve a wide variety of different robot types with little to no modification, while end-to-end approaches tend to be more platform-specific. For this reason, mapping is a crucial area of interest in robotics and this thesis aims to contribute to it.

Many restrictions on the robot's movement are imposed by the geometry of the space the robot is in, for example, in the form of walls or pits. We can sample this geometry using sensors like LiDARs and use different algorithms and representations to estimate its shape. The simplest maps take the form of 2D occupancy grids telling us whether each cell contains an obstacle or is freely traversable. While not requiring a lot of memory to store, being easy to process and interpret, occupancy grids tend to be a bit too simplistic, as an obstacle for one robot can be easily traversed by another, and not all free cells are created equal - a robot can drive on grass, but driving on a paved road is way easier.

To remedy these drawbacks a natural extension of 2D occupancy maps is to store more information in each cell, for example, traversability as a more fine-grained metric rather than just a binary value. By far the most common value to include is height, creating elevation maps. Elevation maps augmented with additional layers of information for each cell are very popular for ground robots, as they contain enough information for robust navigation while being lightweight and easy to work with. Unfortunately, one of their weaknesses is overhangs, as they fail to account for the fact that a robot can pass underneath them. In constructed environments, we can use assumptions on ceiling heights and object sizes to effectively filter overhangs out or use maps with two layers - one representing the ground and the other representing the ceiling. In truly unstructured spaces, like for example in a forest, these techniques tend to fail.

For robustly mapping outdoor environments, we have to extend the map into the third dimension. This has the added benefit of not only being useful for ground robots but for drones and submarines as well, as their movement is not bound to a single plane. Most 3D maps still use the discretized approach, separating the world into a 3D grid, where each cell (called a voxel) contains information about its occupancy and other useful values. Even though voxel maps are more computationally expensive, by running their processing algorithms in parallel on the graphics processing unit (GPU), we are able to make them real-time capable. For these reasons, 3D voxel maps are the best option for the geometric representation of unstructured outdoor environments.

Unfortunately, geometry alone is often not sufficient to safely navigate through an area. A puddle of mud may look like an easily traversable patch of flat ground on a LiDAR scan, and tall grass may appear to be an insurmountable obstacle. Many geometry-only methods try to get around this by anything from simple heuristics on voxel occupancy density to sophisticated neural networks for point cloud semantic segmentation. However, there is a simpler way of figuring out what is what - using a camera. While identifying objects by their shape is a challenging problem, we have reliable and efficient methods of semantically segmenting RGB images.

But there is a catch: Camera images can tell us what objects are and in which direction they are located, but they cannot tell us how far they are. There are methods trying to remedy that: Stereo cameras triangulate the object's position from two separate images taken at the same time. Visual simultaneous localization and mapping (SLAM) uses images from different times and known robot poses to do the same. Some methods just try to learn the depth probabilities for each pixel

using neural networks. However, these often cannot reach the accuracy obtained by measuring the positions of objects directly with a LiDAR or a depth camera. Additionally, cameras and LiDARs have different data density, range and performance in certain environments - LiDARs are good in the dark, while cameras are unaffected by dust. Each modality excels in one area and lacks in the other. We need to leverage their complementary advantages to design a good mapping system. The difficulty arises from the fact that the geometric and semantic information most often comes from separate sensors, their representations are inherently incompatible, and they need to be aligned to be included in the same map. This can be achieved in one of three ways:

- At the sensor level - projecting the point cloud onto the image, giving each point in the field of view of the camera a semantic label to be then included in the map.
- At the map level - projecting rays out of each pixel in the image and placing each pixel's semantic label into the first voxel each ray encounters.
- Independently - we can estimate the depth for each pixel using machine learning and then assign the labels to the corresponding voxel.

Fusing semantic and geometric information at the sensor level is challenging because we need to make sure the point cloud and the image have been taken at the exact same time otherwise they would be misaligned. This is possible for two sensors, but each additional one makes the task more and more complicated. Because the geometric voxel map is aggregated over time, synchronization is not an issue. However, because a voxel map is a coarser discrete representation of continuous 3D space than a point cloud, simple projection encounters issues with occlusions and aliasing, causing, for example, the edges of objects being labeled incorrectly - the labels “bleeding” into each other. Machine learning allows us to learn complex relationships between inputs and outputs, such as the one between what we see in an image and how the corresponding voxel map should be labeled. Unfortunately, current neural network-based methods for depth estimation [1], [2], [3], [4] tend to be dependent on the intrinsic parameters of the camera used, requiring a significant amount of work to apply them to a different platform.

1.2 Related work

The classical approach to mapping is to use a *2D occupancy map*, where each cell has information about whether it is an obstacle or freely traversable space [5], [6]. The straightforward way to use this approach is to classify each cell in a fixed class - obstacle, free, or unknown [5]. While maps like these are easily interpretable, they might cause issues due to oversimplification or measurement errors. Storing the probability that each cell is occupied is a way of more accurately representing the knowledge we have about the environment [6].

2D occupancy maps are a good way of representing simple, flat environments, however, once more complexity is introduced they become insufficient. The natural extension to deal with increased complexity is to store more information in each cell than just the occupancy, creating a *multi-layered costmap* [7], [2]. The additional information can, for example, be the estimated traversability cost or a classification of different obstacle types. One particularly important piece of information to include is the terrain elevation in each cell [8], creating an *elevation map*. Elevation maps are an easy way to remove the implicit constraint on reasonably flat terrain present in standard 2D occupancy maps by making them 2.5D.

Another common type of a 2D map is a *bird's eye view* (BEV) map containing traversability estimations [3], [4], [1]. They are popular for fusing geometric and semantic information using machine learning because they can be implemented to be differentiable and used downstream in an end-to-end learned pipeline [1], [9].

2D traversability maps or 2.5D elevation maps are the mapping representations of choice for most ground robots, but they are severely limiting for robots operating in 3 dimensions - drones and submarines. Additionally, complex, unstructured outdoor environments may be hard to capture without the third dimension. This is where *voxel mapping* methods come in [10], [11], [12]. Like 2D maps, they discretize the environment, but they discretize it into 3D cubes called voxels instead of just 2D cells. Their greatest weakness used to be long computation times [12], but that has been solved by parallelization on the GPU. Voxel maps can also be “multi-layered” including more than just occupancy information in each voxel, for example, heuristics about the properties of the voxel [10] or signed distance field values allowing sub-voxel accuracy in the reconstruction of the environment [11].

Another popular method for full 3D mapping is to use *octrees* [13] separating the world into a set of nested cubes. Each cube contains 8 smaller cubes, which each in turn contains 8 even smaller cubes and so on. These methods predict the occupancy probability of each cube, giving us a representation of the geometry of the environment with on-demand variable accuracy, which is useful for quick information retrieval from the map while preserving detail.

All of the above-mentioned mapping methods discretize the world and try to represent it by the properties of each cell. In recent years, machine learning methods of representing 3D scenes using neural networks called *neural radiance fields* (NeRFs) [14] have become popular and have been adapted for use in robotic mapping [15], [16]. While these methods can surpass the state of the art accuracy and are capable of real-time data retrieval [17], they struggle with the time it takes to insert new information because that requires retraining the networks representing the map, meaning they are incapable of being used in real time.

Sometimes the creation of a map is only an intermediate step for deciding which action the robot should take, and as map creation tends to be time-consuming and memory intensive, there are methods trying to go straight from sensor measurements to robot commands [1], [18]. While these methods tend to be fast and compact, the maps created by mapping algorithms can be used in other tasks than just planning.

Mapping the geometry of the environment alone is often not sufficient to plan robust paths, therefore many methods exist to include semantics from images in both for 2.5D elevation maps [19], [20], [21] and 3D voxel maps [22]. However, these methods use either painted point clouds from RGB-D cameras [21], [22] or simple projection using the camera intrinsic parameters [19], [20], and suffer from the associated issues (see Section 4.1.1). To get around this they implement sophisticated probabilistic methods for merging the newly acquired semantic information with the one already present in the map, namely bayesian inference [23] or exponential averaging [19] to update label probability distributions rather than just storing one label in each cell. It is important to note, that [19] also includes fusion algorithms and other support for more data modalities, not just semantic labels.

1.3 Contribution

The goal of this thesis was to design a pipeline capable of creating a voxel map out of a point cloud and labeling each voxel according to an image where each pixel has an associated semantic class while running real-time onboard a robot.

Specifically, the contributions are:

- G-SVOM: A GPU Accelerated Semantic Voxel Off-Road Mapping System, an extension to G-VOM [10] allowing for the inclusion of semantic labels from segmented images into the map. To show the real-world applicability of G-SVOM, we ran it on sensor data recorded from a payload box meant for the ANYbotics ANYmal D legged robot.
- A novel neural network-based method of semantic label to voxel association, which uses the geometric context from the map to mitigate issues associated with simple projection, increasing the accuracy of voxel labeling compared to the best performing benchmark by 10.2 % when the map is being labeled from just a single image, and by 5.1 % when labeling a map from a series of images from different poses. A notable improvement is in the number of voxels labeled: While labeling a map from a series of different poses, simple projection labels only 87.7 % of voxels due to occlusions, but our method labels 95.0 % of them. The method is designed in such a way, that the trained model is independent of the intrinsic parameters of the camera.
- G-SVOM including the semantic label to voxel association method are publicly available and open-source¹.

¹www.github.com/leggedrobotics/G-VOM/tree/dev/hzamazal/G-SVOM/ml_model

Chapter 2

Method

2.1 G-VOM

2.1.1 G-VOM explained

G-SVOM uses G-VOM as its voxel mapping backbone. The main reason for this choice out of the many voxel mapping pipelines available is its clearly coded and easily modifiable open source python implementation. This section contains a simplified overview of how it works, focusing on concepts crucial for understanding the semantic labeling method. To get a full explanation of G-VOM, please read the paper by its authors, Timothy Overbye and Srikanth Saripalli [10].

G-VOM discretizes the entire world into a 3D grid with the origin of the world coordinate frame w being at the front-bottom-right corner of the voxel with index $(0 \ 0 \ 0)$ and the voxel edges being parallel to the axes of the world frame.

All voxels are of the same size. The voxel edges parallel to the x and y axes of the world frame have the same length s_{xy} and the edge parallel to the z axis has the length s_z . To figure out which voxel index $\mathbf{i}_g = (i_x \ i_y \ i_z)$ a point in the world frame $\mathbf{x}_w = (x_x \ x_y \ x_z)$ belongs to, the following operation is used:

$$\begin{aligned} i_x &= \left\lfloor \frac{x_x}{s_{xy}} \right\rfloor, \\ i_y &= \left\lfloor \frac{x_y}{s_{xy}} \right\rfloor, \\ i_z &= \left\lfloor \frac{x_z}{s_z} \right\rfloor. \end{aligned} \tag{2.1}$$

To conserve memory, only a fixed size cutout of the world grid around the robot is stored as the actual map. This map has the length of l_{xy} voxels along the x and y axes and of l_z voxels along the z axis. To convert between the voxel index in the map $\mathbf{i}_m$ and voxel index in the world grid $\mathbf{i}_g$, we use the index of the voxel with the lowest x, y and z indexes in the map $\mathbf{o}_g$ called the *map origin*:

$$\mathbf{i}_m = \mathbf{i}_g - \mathbf{o}_g. \tag{2.2}$$

The map creation and maintenance pipeline is shown in Figure 2.1. First, point clouds from different sensors are separately converted into *intermediate maps* and then together with the map from the previous time step fused into a new *combined map*. This combined map is the “map from the previous time step” the next time the intermediate maps are being fused together.

To create an intermediate map, G-VOM needs a point cloud and the robot’s position $\mathbf{x}_w$ at the time the point cloud was taken both expressed in the world frame. The

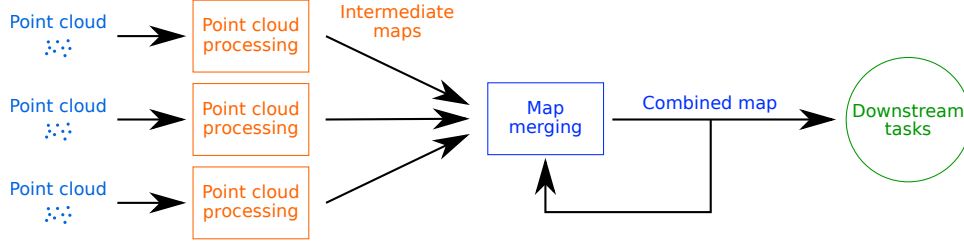


Figure 2.1: The G-VOM map processing pipeline.

map is centered on the robot, so to calculate its origin, the robot’s position is converted to a world grid index $\mathbf{x}_g$ using Equation 2.1 and then shifted by half of the map size:

$$\mathbf{o}_g = \mathbf{x}_g - \left(\left\lfloor \frac{l_{xy}}{2} \right\rfloor \quad \left\lfloor \frac{l_{xy}}{2} \right\rfloor \quad \left\lfloor \frac{l_z}{2} \right\rfloor \right). \quad (2.3)$$

Each voxel inside the bounds of the map defined by $\mathbf{o}_g$, l_{xy} and l_z is marked occupied, if there is at least one point from the point cloud lying in it. For each occupied voxel $\mathbf{i}$, we count the number of point cloud points lying inside it n_{pi} and the number of LiDAR rays passing through it n_{ri} . Rays going to the points lying inside the voxel don’t count towards n_{ri} . These are important to calculate the *voxel density*

$$d_i = \frac{n_{pi}}{n_{pi} + n_{ri}}. \quad (2.4)$$

The idea behind the voxel density metric is, if a ray has to pass through a voxel to measure a point somewhere further away the voxel cannot be fully solid.

During the point cloud processing step, additional heuristics, like the height of the lowest point in each voxel, are also calculated. The intermediate maps can be created in parallel from point clouds from multiple sensors using multithreading and are stored in a time-ordered buffer for map merging.

When the intermediate maps are merged to create the combined map, the last entry into the buffer (closest in time to the start of the map merging process) is used for the combined map’s location. We iterate over the buffer, and wherever voxels from an intermediate map overlap with the combined map, we add their point and ray counts to the counts in the combined map. Other metrics can be merged in other ways, for example the minimum height of a point in each voxel always picks the smaller value between the combined map and the newly introduced intermediate map.

After all the buffered intermediate maps have been merged, it is time to merge the previous combined map. The process is identical to merging another intermediate map, except for one difference: removing voxels. When merging the intermediate maps, a voxel is marked as occupied in the combined map if it is occupied in any of the intermediate maps because we sum the point and ray counts. That is not so when merging the previous combined map. If a voxel has more than 10 rays passing through it in the new combined map, it can never be marked as occupied, no matter how many points lie in it in the old combined map. This is to help remove voxels previously occupied by moving objects.

It is important to mention how G-VOM maps are represented: using a lookup table and value buffers. The lookup table contains an entry for each voxel inside the map. If a voxel is occupied, the lookup table contains an index telling us where to find the information about said voxel (the point count n_{pi} , the ray count n_{ri} etc.) in the buffers. If a voxel is unoccupied, the lookup table contains a negative number $-1 - n_{ri}$ representing how many rays pass through the voxel. This lookup table and

buffer approach reduces the amount of memory required to run G-VOM because we only need buffer slots for occupied voxels.

2.1.2 G-VOM modifications

To create ground truth semantically labeled voxel maps for training and evaluation of the semantic label to voxel association methods, we had to add certain functionalities to G-VOM.

The obvious one is the ability to handle semantic labels. An entry for a semantic label in the form of a vector of specified length was added to each voxel, where the zero vector $\mathbf{0}$ represents *unlabeled*. For this thesis, we used only single number labels. Because our training dataset consists of perfect semantically segmented depth images (see Section 3.1), we chose point cloud painting for the creation of ground truth voxel labels. We added three additional inputs to the intermediate map creation function - semantically segmented image, its intrinsic camera matrix and pose in the world frame. We use those to project the point cloud onto the image, providing each point with a semantic label. Each voxel then receives the same semantic label as the majority of points within it. During the map merging step, the label from the newer map replaces the label from the older map. The only exception to this rule is if the voxel in the newer map has the label 0 - *unlabeled*. In that case the old label takes precedence.

We don't want to run G-VOM online during training, we want to create the ground truth voxel maps in advance. That would not be possible when the map is a rolling window around the robot. That is why we added the option for the combined map to change size and grow to contain everything the robot ever saw, while the intermediate maps still keep the fixed size constraint.

While we want to create the entire map in advance, we still want to be able to simulate its growth to give the model training samples accurate to the actual use cases. To do that, we added one more metric to each voxel - the time when it was first observed to be occupied. Then when sampling the map for training, we can filter out the yet-to-be-seen voxels based on the timestamp of the sample. This approach has one drawback: if a voxel is removed at any point, the final map will never know it was ever occupied. This is not a problem because our training dataset does not contain any moving objects.

2.2 Semantic label association

As G-VOM handles all of the geometry of the map, the main focus of this thesis is the method of associating semantic labels from images with the occupied voxels. The exact task is as follows: We have an unlabeled voxel map and an image from a camera with known intrinsic and extrinsic calibration parameters. Our goal is to assign each voxel in the field of view of the camera a label corresponding to the object occupying said voxel based on what is in the image.

From the pinhole camera model we know, that the voxel (or voxels) containing the label in a pixel lies somewhere along the ray defined by the camera center and said pixel. The question of the label association problem is, which voxels along the ray should be labeled. That is why all our methods operate on single pixel-ray pairs. This has the benefit of being able to easily work on images of different sizes or sampling only individual pixels. Additionally, because we project the ray and extract the occupancy (density) of the voxels separately from our label association methods, these methods are independent of the intrinsic camera calibration parameters. Here it is important to note, that even though the label could theoretically be placed

anywhere along the ray, we look along it only for a fixed distance of N voxels away from the camera center.

2.2.1 Benchmarks

Before we attempt to solve the problem using machine learning, it is important to know, how good of a result we can get using only simple rules. To do that, we designed three benchmark methods we can compare our other solutions to.

First voxel labeling

The first voxel labeling benchmark assigns the semantic label to first voxel along the ray (closest to the camera center) which has the density higher than a fixed threshold $d_i > t_d$ (see Figure 2.2). To learn how the threshold t_d has been selected see Section 3.2.1. The idea behind this heuristic is simple - most things are not see through, therefore the label in a pixel should belong to the first object a ray projected from the pixel encounters. However, if a voxel is not densely occupied, there is a higher chance of the camera actually seeing some other object behind it, so we assign the labels only to sufficiently densely occupied voxels.

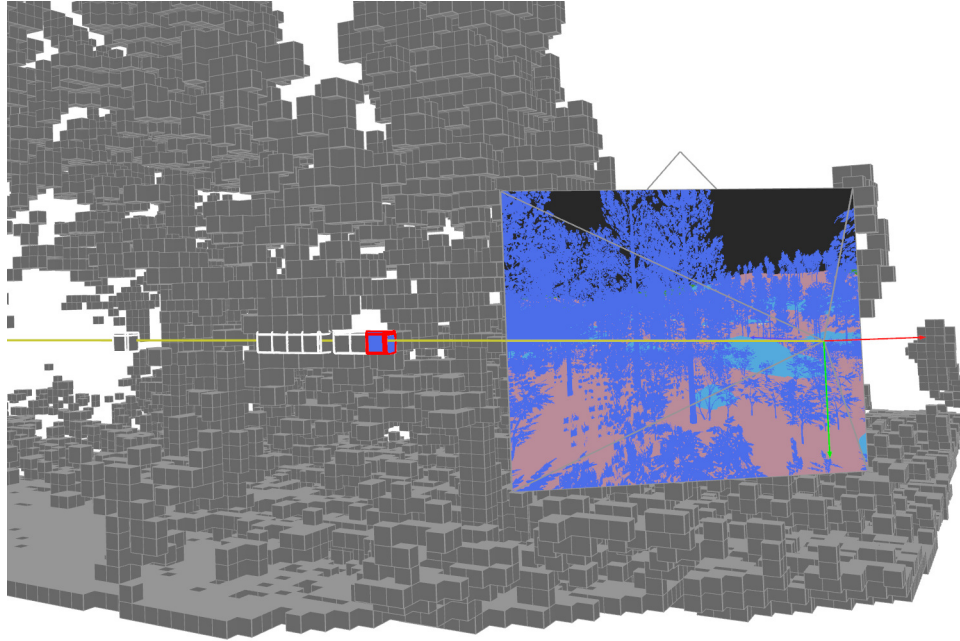


Figure 2.2: The first voxel benchmark assigns the label to the first voxel the ray encounters with density higher than a threshold.

First group of voxels labeling

The first group of voxels labeling benchmark assigns the semantic label to the first (closest to the camera center) continuous group of voxels with the density higher than a fixed threshold $d_i > t_d$ (see Figure 2.3). It is an attempt at solving an issue the first voxel labeling benchmark has with occlusions. Usually, voxels right next to each other belong to the same object, but because the first voxel labeling benchmark labels only the first voxel it encounters, a large number of voxels are never observed and therefore never labeled.

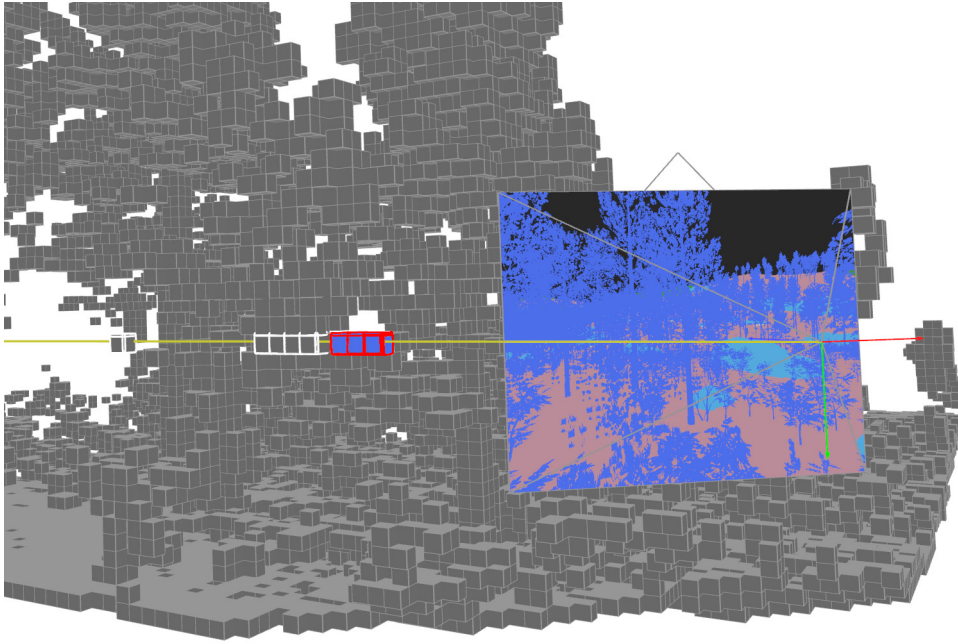


Figure 2.3: The first group of voxels labeling benchmark assigns the semantic label to the first continuous group of voxels with density higher than a threshold.

All voxels labeling

The all voxels labeling benchmark assigns the label to all voxels lying along the ray without any regard for their density (see Figure 2.4).

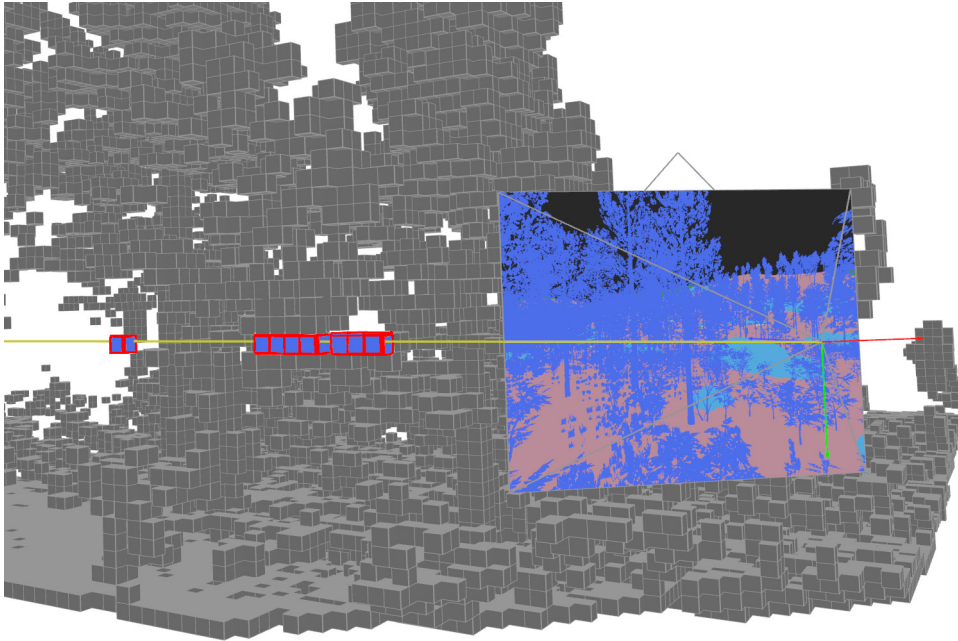


Figure 2.4: The all voxels labeling benchmark assigns the label to all voxels lying along the ray.

2.2.2 Learned methods using density along the ray

The first group of machine learning-based models has the same input information available as the benchmarks: the *semantic label* of the pixel expressed as a one-hot encoded vector $\mathbf{s} \in \mathbb{R}^M$, where M is the number of possible semantic labels, and the *densities of voxels along a ray* projected from the pixel as a vector $\mathbf{d} \in \mathbb{R}^N$, $\mathbf{d} = (d_1 \ d_2 \ \dots \ d_N)^T$, where the element d_1 corresponds to the closest voxel to the camera center and the element d_N to the one furthest away. As an output, we want a *label mapping vector* $\mathbf{y} \in \mathbb{R}^N$ of the same dimension N as the densities vector $\mathbf{d}$, where each entry tells whether to place the label from the input pixel into the corresponding voxel along the ray.

We use the average per-voxel binary cross entropy loss to train the models because the outputs are binary *place/do not place*. We can place a label only into occupied voxels, therefore when calculating the loss we use the input voxel densities vector as a mask to average the loss only over the voxels with density larger than 0. Even though the binary cross entropy loss works only for network outputs between 0 and 1, our models have no layers on the output to guarantee that is the case. The reason for this is we use the PyTorch [24] implementation `BCEWithLogitsLoss`, which includes a sigmoid function before the loss calculation. It is implemented this way for greater numerical stability.

Unlike the benchmarks, the machine learning models don't output a simple boolean value for each voxel but rather a floating point score. We label those voxels that have their score higher than a certain threshold, which is selected in order to maximize the model's labeling accuracy (see Section 3.2).

Model V1: Simple MLP

The first neural network we designed is the simplest one - we stack the semantic label vector $\mathbf{s}$ and the densities along the ray vector $\mathbf{d}$ into one long input vector $\mathbf{x} = (\mathbf{s} \ \mathbf{d})^T \in \mathbb{R}^{M+N}$ and pass it through 2 fully connected linear layers with a sigmoid activation function in the middle. The first layer keeps the input vector's dimension $M+N \rightarrow M+N$ and the second layer reduces the dimension to get the correctly sized output $M+N \rightarrow N$.

Model V2: Separate semantic label and density vector processing

The second network we designed first preprocesses the two inputs separately and then merges them to calculate the output vector. In the preprocessing step, the semantic label vector $\mathbf{s}$ passes through a linear layer $M \rightarrow N$ increasing its dimension to be the same as the density vector's. The density vector $\mathbf{d}$ is also passed through a linear layer but keeping its dimension $N \rightarrow N$. Both of these are then passed through a sigmoid and elementwise multiplied with each other. The result is passed through one more $N \rightarrow N$ postprocessing linear layer.

The architecture is illustrated in Figure 2.5.

Model V3: Discretized voxel density

The previous two models could potentially have one weakness: Voxels with larger density could be favored for label placement because their larger value increases the weight of their corresponding feature vectors in the density vector processing layer. To avoid that in model V3, we round the density value in each voxel to the nearest tenth, providing us with only 11 possible values $(0, 0.1, \dots, 1)$, which can be represented as a one-hot encoded vector for each voxel. Each of these vectors is then passed through a $11 \rightarrow 1$ linear layer and then through a sigmoid activation function, giving us a N long density feature vector. The preprocessing of the semantic label

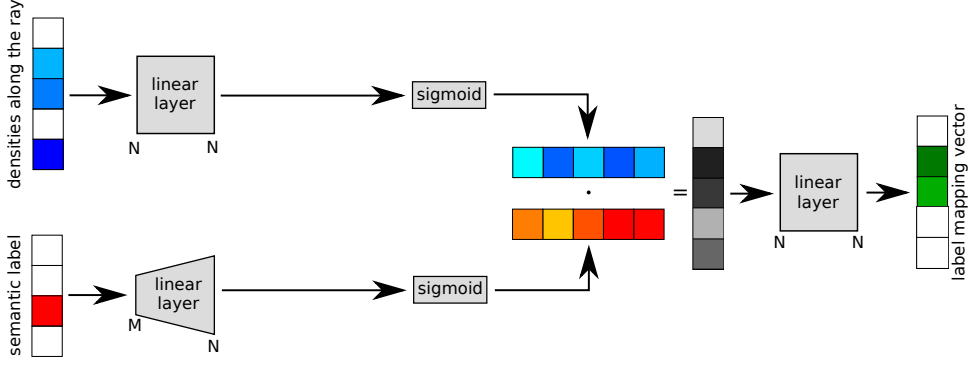


Figure 2.5: Model V2 first processes each input separately, before merging them together using elementwise multiplication and passing the result through a postprocessing layer.

is the same as in model V2, as is the semantics and geometry fusion step and final postprocessing.

The architecture is illustrated in Figure 2.6.

Model V4: Explicit semantic label to voxel density correlation

While model V3 prevents denser voxels from being automatically preferred for label placement, it cannot account for different labels having different voxel density preferences. To allow for this in model V4, the semantic label input vector $\mathbf{s}$ is passed through two separate linear layers: The first one is identical to the semantic label processing layer in models V2 and V3 and followed up by a sigmoid function, producing an N long label depth preference vector. The second one converts the M long one-hot encoded semantic label into an 8 long semantic feature vector $\mathbf{f}_s$. The voxel densities get discretized the same way as in model V3 and passed through an $11 \rightarrow 8$ linear layer to get a density feature vector $\mathbf{f}_{di}$. Then for each voxel, we calculate the dot product between the semantic feature vector and its density feature vector $\mathbf{f}_{di}^T \cdot \mathbf{f}_s$ and pass it through a sigmoid activation function to get a N long semantics and density correlation vector. This semantics and density correlation vector is added to the label depth preference vector, the result passed through a $N \rightarrow N$ linear postprocessing layer, same as for the previous two models.

The architecture is illustrated in Figure 2.7.

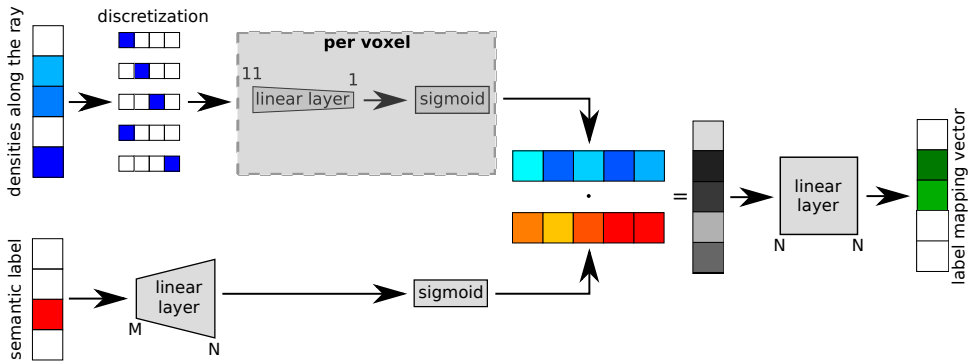


Figure 2.6: Model V3 discretizes and one hot encodes the voxel densities, before merging the geometric information with the semantics.

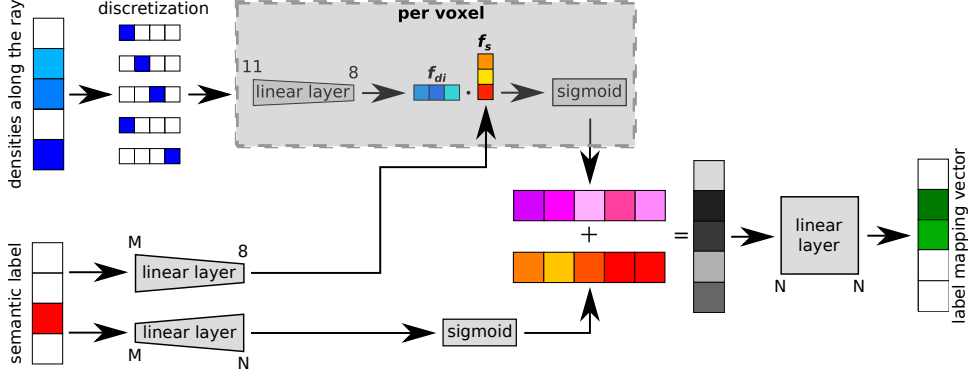


Figure 2.7: Model V4 allows for an explicit semantic label to density correlation through the density feature vectors f_{di} and semantic feature vectors f_s .

Model V5: Ray direction and positional embedding

Models V2 - V4 try to improve upon model V1 by modifying the network architecture. Model V5 attempts to increase the performance by using more of the easily accessible information, namely the *direction of the projected ray in the world frame* and a *positional embedding* for each of the voxels along the ray.

For calculating the ray direction, we know the pose of the camera in the world frame and therefore also the rotation matrix R_w^c rotating a vector in the world frame v_w to the camera frame $v_c = R_w^c v_w$. In the real world, measuring the yaw of a robot is significantly harder than the pitch and roll because changes in yaw don't effect the measurement of gravitational acceleration. For that reason, we decompose the rotation matrix R_w^c into the rotations around the three principal axes in the yaw (rotation around the z axis), roll (rotation around the x axis) and pitch (rotation around the y axis) order, set the yaw rotation to 0 and assamble the matrix again. This gives us the R_w^{c*} matrix representing the rotation from the world frame to the camera frame, ignoring yaw.

The direction of the ray is not only defined by the camera, but also by the pixel the ray is projected from. We can find the rotation matrix representing this orientation R_{c*}^r by first identifying the matrix R_{c*}^m , rotating a point from the camera frame $c*$ to an intermediate frame m , the y axis of which is parallel to the y axis of the camera frame and the z axis is rotated to point in the direction of the pixel column the desired pixel is in. Then we find the rotation matrix R_m^r from the intermediate frame to the ray frame r , which has the x axis pointing in the same direction as the intermediate frame, and the z axis is pointing in the direction of the ray. The schema of the problem with all the important values is shown in Figure 2.8.

If we have an intrinsic camera matrix

$$I = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (2.5)$$

and a pixel $p = (u \ v)$, we can get the angle θ_y we have to rotate the camera frame $c*$ around its y axis by to get to the intermediate frame m as

$$\theta_y = \arctan\left(\frac{u - c_x}{f_x}\right). \quad (2.6)$$

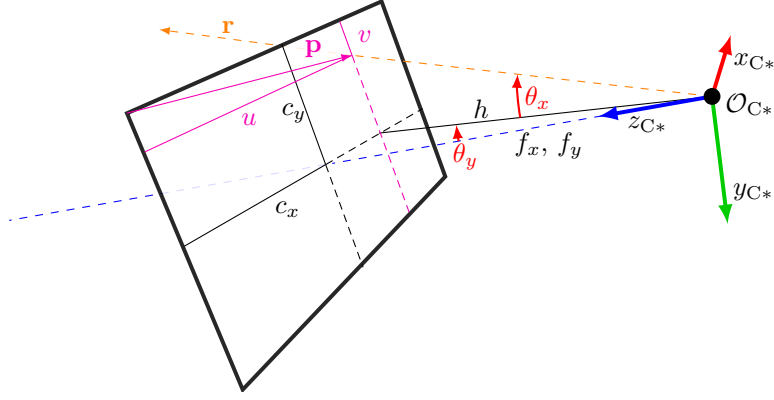


Figure 2.8: A schema of the ray direction in camera frame determination problem.

The rotation matrix R_{c*}^m is then

$$R_{c*}^m = \begin{pmatrix} \cos(\theta_y) & 0 & -\sin(\theta_y) \\ 0 & 1 & 0 \\ \sin(\theta_y) & 0 & \cos(\theta_y) \end{pmatrix}. \quad (2.7)$$

To find the rotation matrix R_m^r from the intermediate frame to the ray frame, we first need to find the distance from the camera center to the image plane along the z axis of the intermediate frame h expressed in units of the vertical pixel size

$$h = \frac{f_y}{f_x} \sqrt{u^2 + f_x^2}. \quad (2.8)$$

We then use it to find the angle θ_x by which we must rotate the intermediate frame m about its x axis to get the ray frame r :

$$\theta_x = \arctan\left(\frac{v - c_y}{h}\right), \quad (2.9)$$

from which we can then assemble the desired rotation matrix

$$R_m^r = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_x) & \sin(\theta_x) \\ 0 & -\sin(\theta_x) & \cos(\theta_x) \end{pmatrix}. \quad (2.10)$$

The resultant rotation matrix representing the orientation of the ray in the world frame (while ignoring the camera yaw) R_w^r is

$$R_w^r = R_m^r \cdot R_{c*}^m \cdot R_w^{c*}. \quad (2.11)$$

To pass this matrix as an input to the neural network, we reshape it into a 9 dimensional vector $\mathbf{r}$.

The sinusoidal positional embedding is inspired by "Attention is all you need" [25]. The value p is added to every element in the densities vector $\mathbf{d}$:

$$p(i) = \begin{cases} \sin(i), & \text{if } i \text{ is even} \\ \cos(i), & \text{if } i \text{ is odd.} \end{cases} \quad (2.12)$$

i is the index of the element in the densities vector the positional embedding value is being added to.

The model V5 itself is the same MLP as model V1, except the input vector length (and therefore also the output length of the first linear layer) is $M + N + 9$.

2.2.3 Learned method using geometric context

To explain a drawback the above described methods have, it is interesting to explore how a human would solve the semantic label to voxel assignment task. When we have a label, say "tree", and a list of candidate voxels, we look at the surroundings of each candidate voxel and use our knowledge of how trees are shaped to decide if the label belongs to the candidate voxel or not. Models that only know the densities of the voxels along the ray cannot do this, as they lack the sufficient geometric context to recognize shapes well. The next model, **model V6**, tries to remedy this issue.

We still take the one-hot encoded semantic label $\mathbf{s}$ from a pixel as one input, and project a ray of fixed length N voxels into the map from it. Instead of just logging the densities of each voxel along the ray, we extract a $K \times K \times K$ cube of voxels around each occupied voxel as its *geometric context* (see Figure 2.9). Each of these geometric contexts is then processed independently by a *geometric feature extractor* to get a *geometric feature vector* $\mathbf{f}_{gi} \in \mathbb{R}^G$. The geometric feature vector of unoccupied voxels is the zero vector $\mathbf{0}$. The geometric feature vectors for each voxel along the ray get passed to the model as a second input. The third input is the ray direction as described in Section 2.2.2.

Before the geometric features are input into the model, they are stacked into one $N \cdot G$ dimensional vector and have the sinusoidal positional embedding from Equation 2.12 added. It is important to note, as the positional embedding is determined by the voxel position along the ray, all G elements of the i -th voxel's geometric feature vector have the same $p(i)$ value added to them. Afterwards, the semantic label vector and the ray direction vector are attached to the geometric features vector to create a $M + N \cdot G + 9$ long input vector. This vector is passed through model V6 itself. It is a 3 layer multilayer perceptron with sigmoid activation functions inbetween the layers, where each layer decreases the length of the vector $M + N \cdot G + 9 \rightarrow 1024 \rightarrow 512 \rightarrow N$.

The schema of how model V6 is used can be seen in Figure 2.10.

While the label association model and the geometric feature extractor are two separate modules that can be changed independently of each other, they are trained together in a single loop with gradients calculated to optimize the extracted geometric features for the best semantic label association. For this reason, it is important to use for inference the weights for the label association model that have been trained together with the used feature extractor. The training loss is the exact same as for the other models: binary cross entropy loss averaged over all the occupied voxels along the ray.

Geometric feature extractors

We designed two geometric feature extractors: MLP-based one and a 3D convolution-based one.

For the MLP-based feature extractor, we reshape the $K \times K \times K$ cube of voxel densities extracted from the map into a vector of dimension K^3 and pass it through a $K^3 \rightarrow 256$ linear layer, a sigmoid activation function and another $256 \rightarrow G$ linear layer to get the geometric feature vector $\mathbf{f}_{gi}$.

The convolution based feature extractor consists of two convolutional layers and one fully connected layer. The first convolutional layer has a kernel size 3, 16 output channels, stride 1 and padding 1. It is then followed by a sigmoid activation function and a max pooling layer of kernel size 3 and stride 3. The second convolutional layer has a kernel size 2, 32 output channels, stride 1 and no padding. It is again followed by a sigmoid and a max pooling layer, this time with kernel size 2 and stride 1. The result is reshaped into a vector and passed through a linear fully-connected layer resulting in the geometric feature vector $\mathbf{f}_{gi}$.

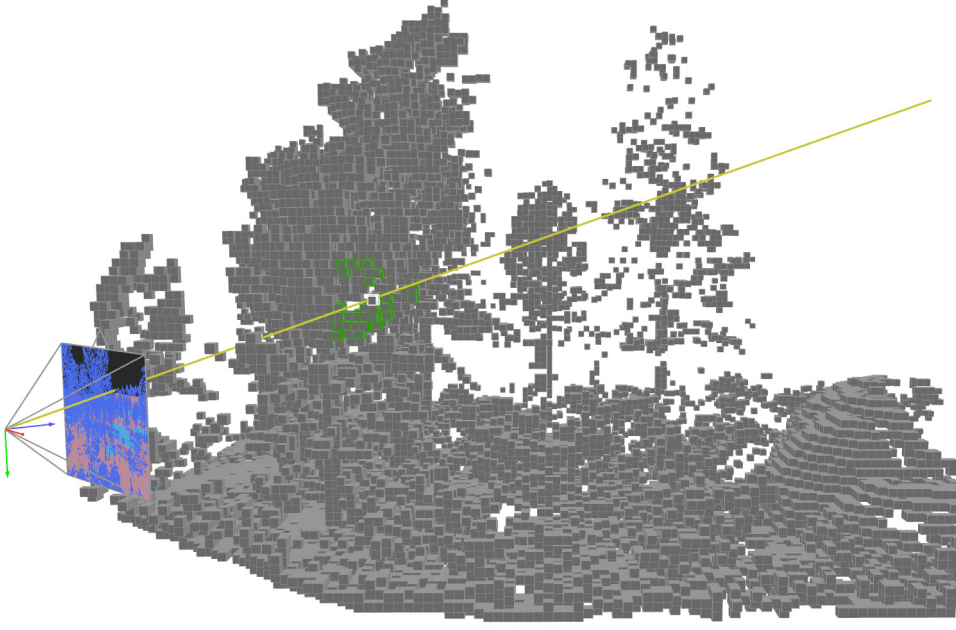


Figure 2.9: For model V6 we extract a cube of voxels (highlighted green) around each occupied voxel along the ray (highlighted white) as a geometric context.

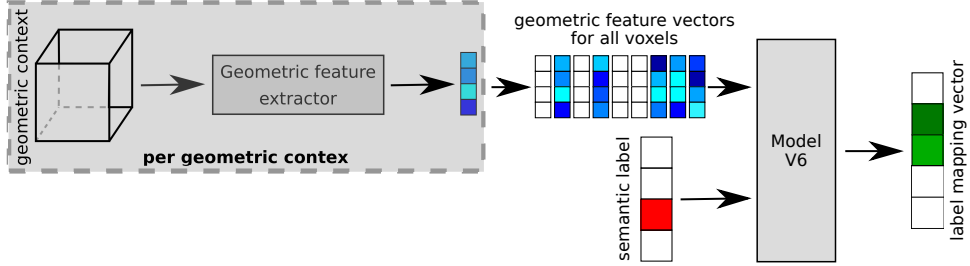


Figure 2.10: First the geometric contexts for all occupied voxels are processed into geometric feature vectors and then passed through model V6 to identify into which voxels the label should be placed.

2.3 G-SVOM architecture

Now that we have outlined the available semantic label to voxel association methods we have to use them within the voxel mapping pipeline. For that purpose, a function to update the semantic labels of the combined map from an image was added. The G-SVOM voxel map creation and maintenance pipeline is shown in Figure 2.11.

The semantic labels update function takes in a semantically segmented image, its intrinsic camera matrix and a coordinate transformation from the camera frame to the world frame. It samples either all pixels in the image, or skips a specified number of rows and columns to subsample the image and reduce the memory requirements and computational time of the process. For each sampled pixel, the relevant model inputs are extracted from the image and the map, and the model is queried to get the pixel-voxel correspondences based on which we label the map. It is important to note, that pixels with the label 0 (*unlabeled*) are ignored, meaning no voxels with a label will be overwritten to having no label. To speed up the process, all per-pixel operations are executed in parallel on the GPU, and the geometric context

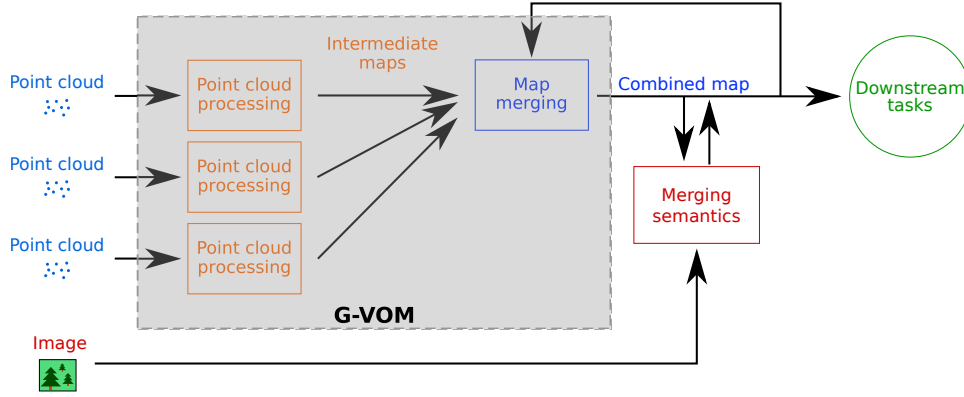


Figure 2.11: The G-SVOM map processing pipeline is identical to G-VOM, except it has the semantic label merging function added. It is also important to note, that while this schema includes the map merging function inside the G-VOM geometric pipeline, it is modified to support merging maps with the semantic labels.

extraction and featurization (if needed for the selected model) is parallelized per-voxel on the GPU.

To reduce the memory complexity of geometric feature extraction, G-SVOM first projects the rays from all the selected pixels to identify the location of each occupied voxel and to which ray it belongs. It then extracts the geometric contexts. This is done so that we can allocate the memory for storing the geometric contexts only after we know how many contexts are required. While doing the raytracing, we need arrays to store the occupied voxels' location and ray assignment, but we don't know in advance how many entries these arrays should have (because that is what we are trying to figure out). So to allocate the arrays, G-SVOM assumes that at maximum 40 % of the voxels the projected rays encounter will be occupied.

Usually, rays from multiple pixels pass through each voxel, therefore it is possible several different labels are supposed to be assigned to the same voxel. In that case the label is selected at random due to the per-pixel parallelized nature of the label assignment operation.

During the map merging process, the semantic labels from a newer map overwrite the labels from the older map, except if the label in the new map is *unlabeled*. In that case the old label is kept.

Chapter 3

Implementation details

The thesis was implemented in two git repositories: G-SVOM itself¹, which is an open sourced fork of the G-VOM repository, and a private repository for development of the semantic label to voxels association methods described in Section 2.2. The development repository of these methods is dependent on the G-SVOM implementation, as it uses modified G-VOM (described in Section 2.1.2) to build the ground truth maps and contains scripts running G-SVOM for testing without the need for a ROS [26] interface. The G-SVOM repository contains everything necessary and works standalone, however, for reasons described in Chapter 4, only the benchmarks and models V1 and V6 are available for semantic label association². The project is written in Python, using the PyTorch [24] library for machine learning, Numpy [27] for other miscellaneous computations, Numba [28] for parallelization on the GPU and OpenCV [29] for working with images.

Most of the decisions regarding the implementation of G-SVOM have been inherited from G-VOM, as our work is just its extension. G-SVOM is a Python class that receives all the parameters, like voxel resolution, intermediate map size and others in its constructor. It also receives the semantics to voxel association model and geometric feature vector extractor this way in a dependency injection scheme. Even though only five semantic association methods are available directly from the G-SVOM repository, all of them have the same interface, so they can be used without requiring any changes to the code if desired. The point cloud processing, map merging and semantics association methods described in Sections 2.1 and 2.3 are methods of the G-SVOM object. It is interesting to note, that if the semantics association method never gets called, G-SVOM functions in the exact same way as G-VOM.

While the point cloud processing and semantics association functions don't return anything, the map merging function returns processed maps, like a positive obstacle and negative obstacle occupancy map or driveable surface elevation map, that can be used downstream in the robotic pipeline. There are also additional debug functions, which can do things like returning a point cloud representing the centers of each voxel. None of these functions take the semantic labels of voxels into account, as they are G-VOM native. The only way the semantic labels are now utilized is in a visualization function, which returns the centers of occupied voxels and their corresponding labels.

For the ease of use within robotic pipelines, G-SVOM has a ROS wrapper. It subscribes to a point cloud, segmented image and camera info topics and publishes the aforementioned preprocessed and debug maps. After receiving a point cloud, it

¹www.github.com/leggedrobotics/G-VOM/tree/dev/hzamazal/G-SVOM/ml_model

²The pretrained weights for models V1 and V6 are included.

builds the intermediate map and stores it in the intermediate map buffer. Every 100 ms, the intermediate maps get merged into the combined map and all the pre-processed and debug maps are published. Whenever a segmented image is received, it is stored, and every 200 ms the semantics from the latest images from each camera get merged into the map, and the labeled point cloud is published. It is important to note, that while receiving data from multiple LiDARs and their parallel processing is theoretically supported by G-SVOM, the ROS wrapper currently supports only subscription to one, and would require modification to subscribe to more. The ROS wrapper is currently configured to receive segmented images from 3 cameras but can work with less, if some of the cameras don't send any images.

The semantic label association repository contains mainly code required for training and testing the label association models and benchmarks: the classes for handling the training and evaluation data, building the ground truth maps, extracting the training samples from them etc. Unlike the G-SVOM repository, which has only one runnable script (the G-SVOM ROS node), this repository stores many different runnable files for all of its various tasks.

3.1 Training and evaluation data

For training and evaluating our models and benchmarks we used TartanAir [30]: A synthetic dataset containing data without any noise from a wide variety of different environments - from urban scenes to forests and mountains. There are several trajectories going through each environment representing a traveling robot with a sequence of samples. Each sample consists of an RGB image, a semantically segmented image, a depth image and a 6 DoF pose. All of these are present twice in every sample, because TartanAir simulates a robot with a stereo camera. We use only the data from the right camera, as the cameras are just 20 cm apart and the data captured by them is mostly identical. The robot that TartanAir simulates is an agile quadcopter drone, so the poses are very varied. The intrinsic camera matrix is also provided.

Not all environments present in TartanAir were suitable for our purpose. We are focusing on mapping of unstructured outdoor environments, therefore we left out the indoor environments, like for example *Hospital*. Additionally, some environments contained moving objects (ex. *Amusement*), which collide with our plans to train and evaluate our models on premade voxel maps (see Section 3.2). An overview of the used environments can be seen in Figure 3.1. We split the environments we want to use into 7 training ones and 6 testing ones, so that we can test our models on scenes that were not present in the training dataset. The environment split is as follows:

- **Training environments:** *Seasonsforest*, *Neighborhood*, *Carwelding*, *Abandonedfactory night*, *Oldtown*, *Westerndesert*, *Gascola*.
- **Testing environments:** *Endofworld*, *Japaneseealley*, *Seasonsforest winter*, *Seasidetown*, *Abandonedfactory*, *Soulcity*.

Table 3.1: The training, validation and test dataset split. We sample 2000 pixels from each TartanAir image (see Section 3.2).

	Training dataset	Validation dataset	Test dataset
TartanAir samples	106 874	26 719	89 813
Pixel samples	213 748 000	53 438 000	179 626 000

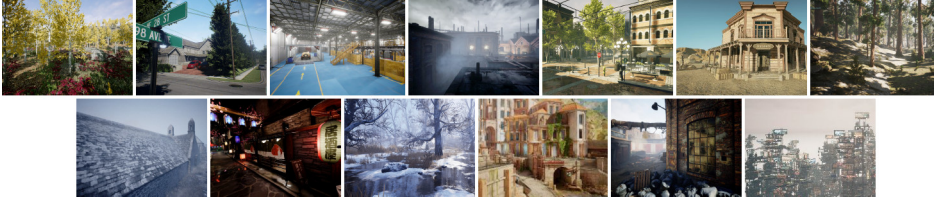


Figure 3.1: An overview of the used TartanAir environments covering a wide range of different scenes. The training environments are in the top row, the testing ones in the bottom row. Source: Wang et al. 2020 [30]

From the environments we use for training we take the first 80 % of samples of each trajectory as the training dataset and the last 20 % as the validation dataset. We use all trajectories and samples in the test environments for testing. The split was not done by randomly picking samples, because consecutive samples are very similar and we want the two datasets to be as different as possible to be representative of real model performance. You can see the dataset sizes in Table 3.1.

Unfortunately, even though TartanAir contains semantically segmented images, the labels provided are not very meaningful as they correspond just to object mesh numbers rather than object types. On top of that, not only do objects of the same type have different labels in different environments, there are also cases where objects of a different type have the same label, sometimes even in the same environment. For that reason, we manually went over all 13 of the environments and created mappings of the TartanAir mesh numbers to the labels present in the COCO-Stuff dataset [31]. These mappings are environment-dependent, therefore two different objects having the same TartanAir mesh number in two different environments are not a problem. However, if two different objects have the same TartanAir mesh number in the same environment we cannot map it to a COCO-Stuff label. Those were mapped to the label 0 (*unlabeled*) and ignored during training and inference.

3.2 Training process and parameters

Before we trained any models, we created and stored the ground truth voxel maps for all trajectories in both the training and testing environments. This was done by using the intrinsic and extrinsic calibration parameters of the depth images to create point clouds and then together with the semantically segmented images (containing COCO-Stuff labels) passing them to modified G-VOM (see Section 2.1.2) to build the semantically segmented ground truth voxel maps.

During training, we need to store all of the pre-made maps in the graphics card’s memory at once. That is not possible with the way G-VOM stores them as the lookup table takes too much space. Luckily, we don’t care about the number of rays passing through unoccupied voxels, we care only about the voxel density of occupied voxels. We can make the map smaller by adding one more entry into the buffer storing data for occupied voxels representing where in the map this occupied voxel is. This then allows us to get rid of the lookup table. If we represent the voxel’s position not by its i_x , i_y and i_z indexes, but by just a single number $i = i_x + i_y \cdot l_{xy} + i_z \cdot l_{xy}^2$, we can sort all of the buffers by this number and use binary search to quickly access a voxel’s data, provided we know its location in the map. This is implemented only for operations on the stored pre-made maps not in G-SVOM itself.

During training, we pick 3 random image-pose pairs from the training dataset and find their corresponding ground truth voxel maps. From each image we randomly sample 2000 pixels, making our batch size $3 \cdot 2000 = 6000$. If any of the sampled

pixels has no label (label 0), it is removed from the batch. The pixel labels are converted to one-hot encoded vectors. Because TartanAir contains only 52 of the 183 COCO-Stuff labels, these vectors have only 52 dimensions (possible one-hot encoded values). The ray out from each selected pixel is traced and either the density vector or the geometric contexts are extracted. During the raytracing process, we also compare the label of each encountered voxel to the pixel’s label to get the expected model output. Afterwards, we pass the inputs in the batch through the trained model, calculate the average binary cross entropy loss for occupied voxels along the rays, and do an optimization step. We use the Adam optimizer [32] with learning rate 0.001 for all models except for model V6 with MLP geometric features, which trained best with learning rate 0.0005.

Table 3.2: The parameters of the maps and models used during training and evaluation.

Symbol	Description	Value
s_{xy}	Size of a voxel along the x and y axes	0.15 m
s_z	Size of a voxel along the z axis	0.15 m
l_{xy}	Size of an intermediate map along the x and y axes	256 voxels
l_z	Size of an intermediate map along the z axis	128 voxels
N	Length of a ray projected out from a pixel	128 voxels
M	Number of possible semantic labels	52 voxels
K	Edge length of a geometric context cube	9 voxels
G	Dimension of geometric context feature vector	16

The parameters of the modified G-VOM voxel mapper and the trained models can be found in Table 3.2. The ray length $N = 128$ was chosen on purpose to be half of the map side length $l_{xy} = 256$ as each intermediate map is placed so that the robot is in its center (see Section 2.1).

The machine learning models V1-V6 output a floating point value, and we need a boolean *place label* or *do not place label* value. As mentioned in Section 2.2.2, we solve this problem by picking a threshold, and if the model output is bigger than this threshold, we place the label. To pick a threshold for each model, we calculated the single view accuracies (see Section 4.1.2) on the validation dataset for thresholds between -5 and 5 at 0.5 increments and picked the threshold with the largest accuracy. Coincidentally, this threshold was 0 for all models.

3.2.1 Benchmark thresholds

Even though the three benchmarks are not neural networks and therefore don’t need to be trained, the first voxel and first group of voxels benchmarks have a parameter to optimize: for them to label a voxel, it has to have a density higher than a certain threshold. To find these density thresholds, we calculated the single view accuracies (see Section 4.1.2) of the two benchmarks on the training environments (meaning on the training and validation datasets together) for thresholds between 0 and 0.9 at 0.1 increments. The graphs showing the dependency of the method’s accuracy on the threshold can be seen in Figure 3.2.

For the first voxel benchmark, we can see the accuracy climbs very slightly from 0 to 0.2, then fairly rapidly drops and steadies itself again for the higher density values. The lower the threshold, the higher the probability of the method labeling one of the first voxels the ray encounters. We can interpret the accuracy curve as the thresholding mechanism being only marginally effective at mitigating wrong assignments. After we pass the 0.3 threshold, we start missing more and more of the visible voxels and the method is making poorer and poorer guesses, rapidly

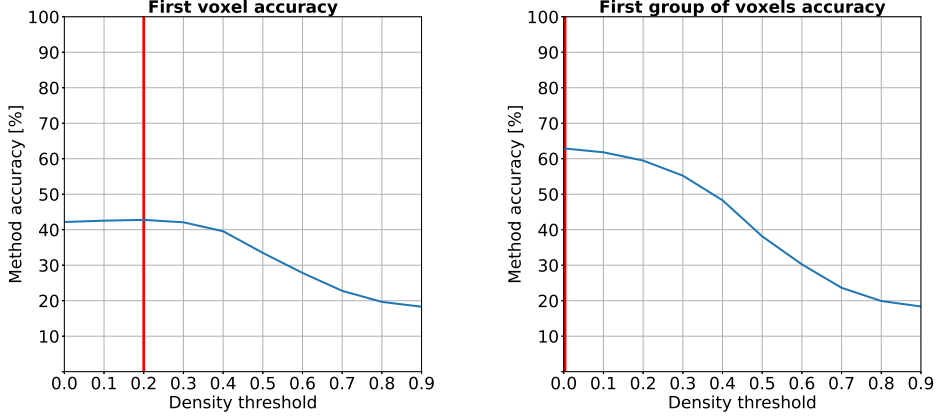


Figure 3.2: The density threshold dependent single view accuracies for the first voxel and first group of voxels benchmarks. The maxima - values corresponding to the selected thresholds - are highlighted by the red lines. The accuracies were calculated on the training environments.

decreasing the accuracy. At the end, most of the voxels are ignored as they have lower densities and it doesn't matter much, if the threshold is 0.8 or 0.9.

The curve for the first group of voxels benchmark supports the theory that the thresholding is not very effective, preferring the lowest threshold possible - just label whatever is in the first voxels the ray encounters. It is consistent with how cameras work - they see the first thing the ray encounters and it is more likely going to be an object closer to the camera.

Table 3.3: The density thresholds for the first voxel and first group of voxels benchmarks.

	First voxel benchmark	First group benchmark
Best density threshold	0.2	0

The best found thresholds can be found in Table 3.3. The voxel densities are represented as floating point numbers, and there is a chance that zero values may not be precisely equal to 0. For that reason, the zero threshold was, both for the threshold finding experiment and later in the first group of voxels benchmark, implemented as 10^{-5} .

Chapter 4

Experiments and Results

This section discusses the experiments we performed to test the presented semantic label to voxel association methods and their inclusion within G-SVOM. We do not explore the strengths and weaknesses of the geometric mapping pipeline, as it has been taken effectively without modification from G-VOM, and its performance is explored and discussed at length in the paper by its authors [10].

4.1 Performance on simulated data

4.1.1 Benchmark analysis

Before we move on to comparing our machine learning models, it is interesting to look at the heuristics-based benchmarks and see their strengths and weaknesses. Figure 4.1 shows a voxel map labeled with each of the benchmarks and the ground truth.

The main issue for the first voxel and first group of voxels benchmark is the fact that they label only the first voxel or only the first few voxels each ray encounters. An example is in Figure 4.2. There are many more voxels the rays encounter, but they are never labeled as they are occluded - hidden behind other objects. This issue is more pronounced for the benchmark that labels only the first voxel and on the other hand, not at all present for the method that labels all voxels.

However, the opposite is true for the other issue the benchmarks face. It is possible, the first voxel a ray encounters doesn't contain the object the camera sees because the voxel map is a discrete approximation of a continuous world and the edges of objects do not necessarily have to align with the edges of voxels. This manifests as the labels "bleeding" into each other around the edges of objects. An example is in Figure 4.3. Thresholding the voxel density is an attempt at mitigating this issue, but if a voxel is occupied only by a small twig, it will never get correctly labeled as it will never pass the density threshold, making some of the voxels not labeled by the first voxel benchmark a bleed issue too.

The reason the all voxels benchmark suffers a lot from the bleed problem is the fact that it labels all of the voxels, even those that definitely cannot be seen from the image. For example, if the ray first encounters a tree and the ground afterwards, the all voxels benchmark labels even the ground as a tree. It is a trade-off between labeling only the first voxel the ray encounters, reducing the guesswork but labeling less voxels, and labeling all voxels but guessing the label for a lot of them. The first group of voxels benchmark is an attempt at finding a middle ground.

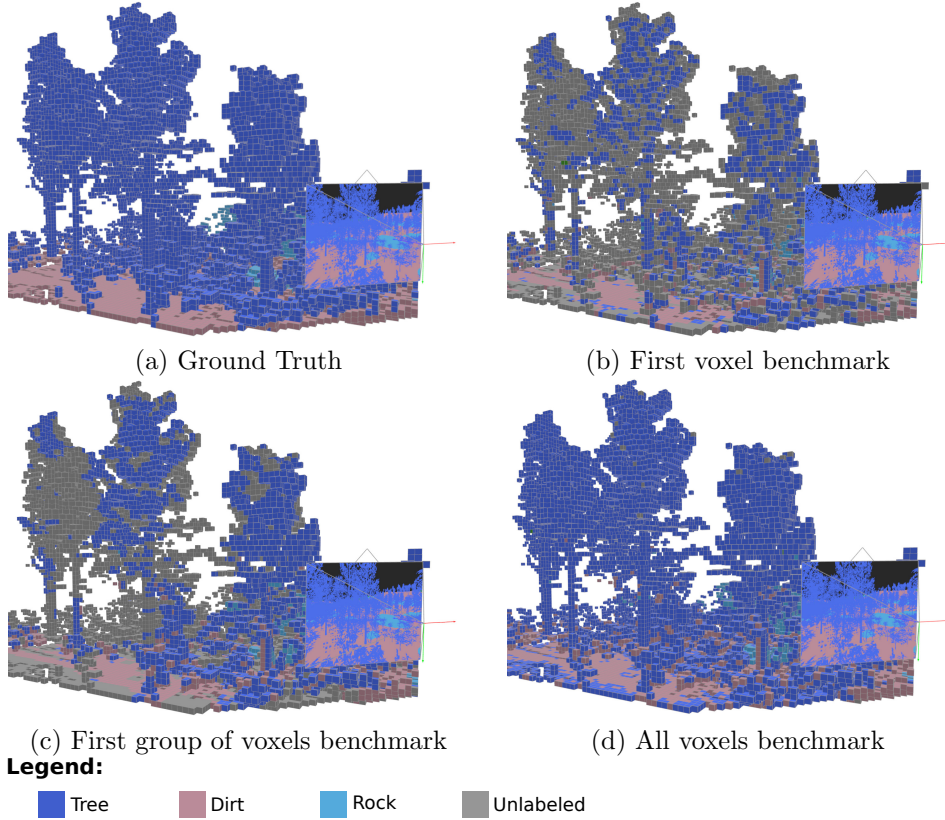


Figure 4.1: A voxel map labeled from a single image by the three benchmarks compared to the ground truth (a). The first voxel benchmark (b) does not label most voxels. The first group of voxels benchmark (c) is better in this regard, but still doesn't label nearly enough of them. While the all voxels benchmark (d) labels most voxels, a lot of those are labeled wrong - for example those along the bottom edge.

4.1.2 Single view accuracy

To compare the performance of our models and the benchmarks, we ran the exact same procedure as we did during training but on the validation and test datasets. Instead of optimizing the model weights, we thresholded the model outputs, and then counted how many entries of the output vectors corresponding to occupied voxels have the same value as the ground truth. The single view accuracy is then

$$a_{\text{single view}} = \frac{\# \text{ of correct entries for occupied voxels in the output vectors}}{\# \text{ of occupied voxels along the rays}}. \quad (4.1)$$

This metric corresponds to taking a single image, using the model to label all voxels within reach (within the field of view of the camera and within the reach of the fixed length rays projected from the pixels), and then checking how many of the voxels were labeled correctly. Voxels that have a label in the ground truth map and the model assigns them no label are considered wrongly labeled, however, voxels with no label in the ground truth map (because of the mismatch between TartanAir semantic labels and real objects - see Section 3.1) are completely excluded from the evaluation. Examples can be seen in Figure 4.4.

Based on the results of this experiment, we see our models significantly outperform

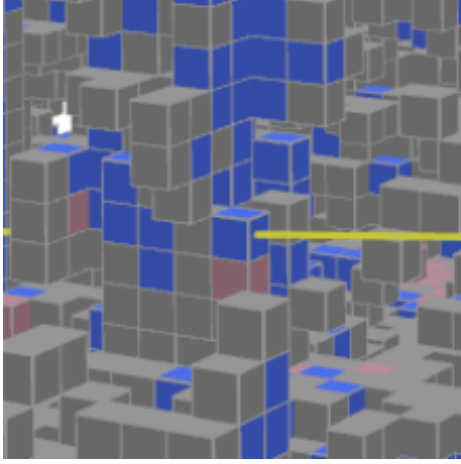


Figure 4.2: Labeling only the first voxel a ray encounters results in unlabeled voxels (grey) due to occlusions.

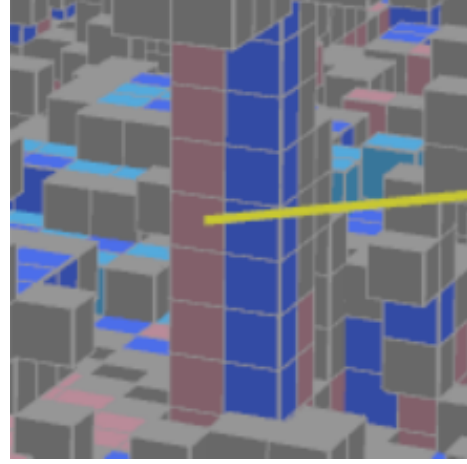


Figure 4.3: The tree trunk is labeled as *dirt* (pink) instead of *tree* (blue) because labeling the first voxel a ray encounters can cause some voxels to be labeled as what the camera sees behind them due to discretization.

Table 4.1: The average model accuracy on the test and validation datasets for labeling only the voxels within the field of view and ray reach of a single image.

Model Version	Validation accuracy	Test accuracy
First voxel benchmark	42.1 %	51.8 %
First group of voxels benchmark	60.1 %	69.0 %
All voxels benchmark	82.2 %	77.1 %
V1	88.0 %	83.4 %
V2	87.4 %	83.5 %
V3	84.6 %	80.7 %
V4	85.3 %	80.9 %
V5	88.2 %	83.7 %
V6 + MLP features	95.1 %	87.3 %
V6 + Convolutional features	94.4 %	86.8 %

the baselines and adding the geometric context has positive effect on model performance. On the other hand, we can see that the architecture modifications of model V2 and additional inputs of model V5 have little to no effect. Models V3 and V4 perform worse than the simple model V1.

We can also observe, that the machine learning-based models perform significantly better on the validation dataset than on the testing one. This has to do with the fact that the validation dataset comes from the same environments as the training data, therefore is closer to what the models have seen during training. This performance boost is not seen with the benchmarks (at least the first voxel and first group of voxels ones) as they have not been trained.

With only this test we lack more in depth information about why each method performs the way it does. For example, does the all voxels benchmark outperform the other benchmarks only because it doesn't suffer from occlusions? To learn more, we need to do additional experiments. As most of our models seem to perform with similar levels of accuracy, we picked only V1 and V6 with MLP features for further

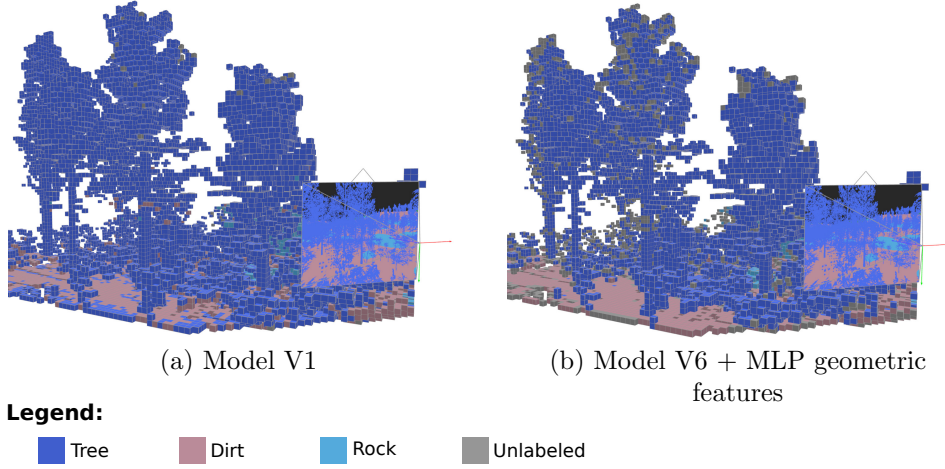


Figure 4.4: A voxel map within the view of a single image labeled by the two significant models. Model V1 labels the vast majority of voxels but suffers from a lot of label bleed. While model V6 labels less voxels, the labels are more often correct, giving model V6 larger overall accuracy. The same test for the benchmarks and the ground truth can be seen in Figure 4.1.

analysis. V1 is the simplest model and adding complexity didn’t seem to improve it much. V6 with MLP features is the best performing model. This is also the reason why only models V1 and V6 with MLP geometric features are available in the open source implementation of G-SVOM.

4.1.3 First voxel accuracy

Table 4.2: The average labeling accuracy of the first voxel a ray encounters in the test dataset.

Model Version	Accuracy of first voxels
First voxel benchmark	81.3 %
First group of voxels benchmark	87.3 %
All voxels benchmark	87.3 %
V1	90.8 %
V6 + MLP features	89.6 %

To evaluate how big of a problem the label bleed around object edges is, we performed the same test of voxel accuracy as in Section 4.1.2, but we compared only the accuracies of the first occupied voxel each ray encountered. These are the voxels that would be directly visible from the image and labeled by the most naïve approach - first voxel labeling without any density thresholding. The accuracies can be found in Table 4.2.

The reason for the first voxel benchmark having a different result from the others is the different density threshold it uses. The lower the threshold, the more of the evaluated voxels are labeled and an unlabeled voxel is always considered wrong, while a labeled voxel has a chance of being right. Because the threshold for the first group of voxels and all voxels benchmarks is 0, their accuracy 87.3% is the exact same as the single view (and also first voxel) accuracy of the most naïve approach (label the first occupied voxel along each ray).

The machine learning-based models have approximately 3 % higher accuracy than the best performing benchmark. What is interesting to observe, is the fact that both of the models have approximately the same accuracy on the first voxel they encounter, even though model V6 with MLP geometric features performs significantly better on the full single view accuracy. This tells us that the difference in performance is mostly on the unseen voxels, supporting the theory that adding geometric context as an input to the model boosts the labeling accuracy of unseen voxels.

4.1.4 Accuracy on labeled maps

So far we have only looked at how the models perform if shown a single image and labeling the voxels within its field of view and reach of the fixed-length rays. However, during deployment G-SVOM will receive a series of images from different robot poses, which it will integrate into the map. The performance on this task may be different than just on voxel labeling from a single image, as viewing a voxel from different angles and distances might allow for better classification, and moving around the map should make more voxels directly visible in the image, reducing the occlusions problem.

To verify this assumption and better evaluate the performance of our methods on the map labeling task, we took the first 3 trajectories in the 6 testing environments and did the following: We took the ground truth voxel map of each trajectory and removed its semantic labels. Then we went through all the samples in the given trajectory, simulating the robot moving through the environment and re-labeling the pre-made voxel map. This way we got a voxel map labeled by our methods independently of the geometric map creation process. Examples of how these maps look can be found in Figure 4.5.

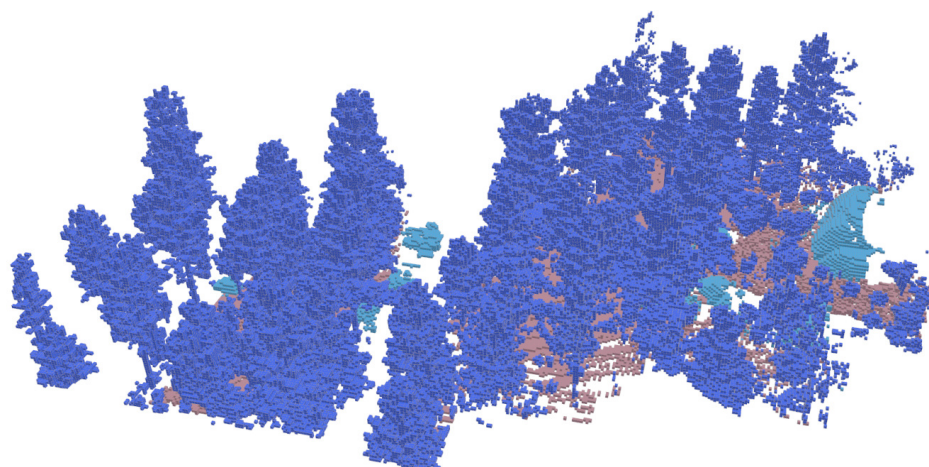
Table 4.3: The average raw labeled map accuracies of the benchmarks and interesting models on the first three trajectories of the test environments.

Environment	First [%]	Group [%]	All [%]	V1 [%]	V6 [%]
<i>Abandonedfactory</i>	70.0	77.1	66.5	74.9	87.2
<i>Endofworld</i>	74.0	87.5	77.7	81.4	92.3
<i>Japanesesealley</i>	69.2	75.5	63.1	75.0	75.5
<i>Seasidetown</i>	75.7	83.4	76.6	79.7	57.8
<i>Seasonsforest w.</i>	62.5	53.6	66.9	79.4	92.7
<i>Soulcity</i>	76.8	82.6	79.9	84.9	84.8
Average	71.4 ± 5.7	76.6 ± 11.6	71.8 ± 9.2	79.2 ± 6.7	81.7 ± 13.2

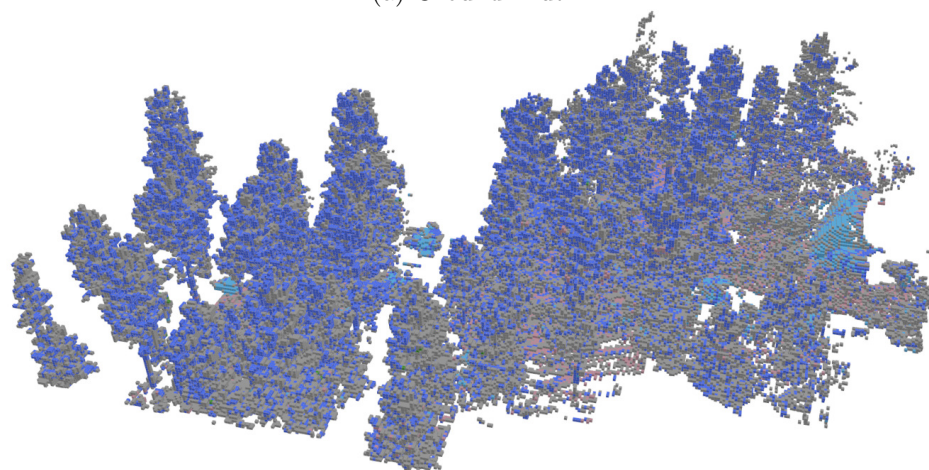
The first metric we measured was the *raw accuracy of the labeled map*: We checked, how many out of all of the voxels in the map have the correct label. Because unlabeled voxels count as wrongly labeled, this can be considered the lower bound on the method’s accuracy. Same as for the single view accuracy, voxels that don’t have any label in the ground truth map are completely excluded from the evaluation. The average raw labeled map accuracies can be seen in Table 4.3.

For calculating the second and third metric, we leverage the fact that we are comparing labeled maps not just model outputs to ground truth. Multiple rays can pass through a single voxel and as our methods operate independently on each ray, we cannot check how each voxel was actually labeled without running the label assignment step of G-SVOM. While this introduces the slight randomness mentioned in Section 2.3, it allows us to more accurately gauge the performance of each method with respect to bleed and occlusions.

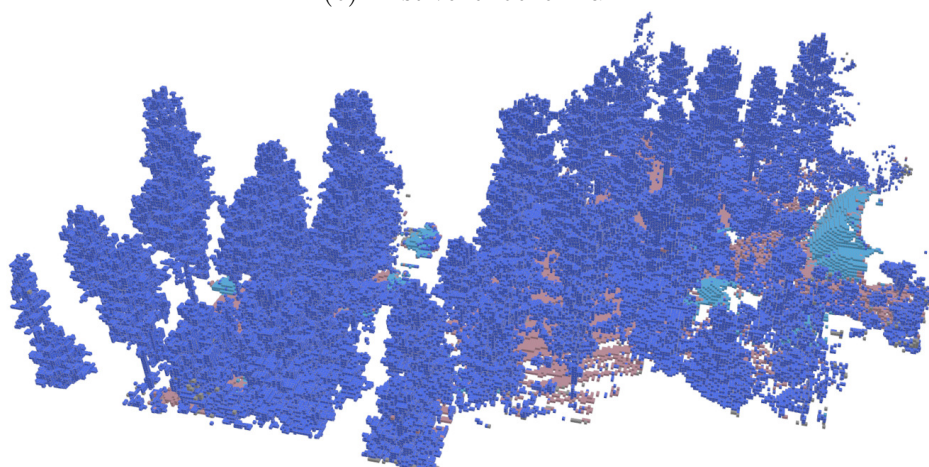
The second metric we measured was the *accuracy of the labeled voxels*. We



(a) Ground Truth



(b) First voxel benchmark



(c) Model V6 + MLP geometric features

Legend:

Figure 4.5: A map of the first trajectory in the *Seasonsforest* environment fully labeled with the first voxel benchmark and model V6. The trained method (model V6) not only labels more of the map, but is also more accurate with the labels it places.

Table 4.4: The average labeled voxel accuracies of the benchmarks and interesting models on the first three trajectories of the test environments.

Environment	First [%]	Group [%]	All [%]	V1 [%]	V6 [%]
<i>Abandonedfactory</i>	78.5	85.4	66.5	75.0	87.6
<i>Endofworld</i>	89.3	95.1	77.7	81.9	95.2
<i>Japanesesealley</i>	78.7	86.8	63.1	75.3	76.9
<i>Seasidetown</i>	86.3	89.0	76.7	70.1	75.0
<i>Seasonsforest w.</i>	70.8	69.8	66.9	79.5	92.9
<i>Soulcity</i>	85.4	91.0	79.9	85.1	86.4
Average	81.5 ± 6.8	86.2 ± 8.7	71.8 ± 7.1	77.8 ± 5.4	85.7 ± 8.2

checked how many of the voxels that got a label assigned got that label assigned correctly. This value is very similar to the raw accuracy, with the only difference being the treatment of unlabeled voxels. While the previous metric considered them wrongly labeled, this one disregards them all together. The average labeling accuracies can be seen in Table 4.4.

Table 4.5: The average labeling coverage of a map labeled by the benchmarks and interesting models on the first three trajectories of the test environments.

Environment	First [%]	Group [%]	All [%]	V1 [%]	V6 [%]
<i>Abandonedfactory</i>	89.3	90.2	99.9	99.8	99.7
<i>Endofworld</i>	82.9	92.0	100.0	99.5	96.9
<i>Japanesesealley</i>	87.9	86.9	100.0	99.7	98.2
<i>Seasidetown</i>	87.7	93.7	99.9	99.5	77.7
<i>Seasonsforest w.</i>	88.3	76.7	100.0	99.9	99.7
<i>Soulcity</i>	90.0	90.7	100.0	99.7	98.0
Average	87.7 ± 2.5	88.2 ± 6.0	99.9 ± 0.0	99.7 ± 0.2	95.0 ± 8.6

The last metric is complimentary to the accuracy of labeled voxels: We want to know the *map coverage* - what fraction of the map got a label assigned without the regard for the correctness of the label. If a method labels only one voxel but does so correctly, it will have a 100% labeling accuracy. However, because the map contains many more voxels than just the one, the map coverage for this toy example would be very low and we would know the method is not as good as it looks. The average map coverage for each model can be seen in Table 4.5.

The trade-off between map coverage (occlusions) and labeling accuracy (bleed) mentioned in Section 4.1.1 is very much present. While the first group of voxels labeling benchmark is the most accurate when considering only the labeled voxels, it provides only 0.5 % higher coverage than the method with the worst coverage of them all - the first voxel benchmark. On the other hand, the all voxels benchmark has the highest average map coverage, but the worst accuracy on the voxels it labels. This trade-off is also present among our trained models. Model V1 provides almost the same map coverage as the all voxels benchmark, but is less accurate than model V6. On the other hand, model V6 has accuracy of labeled voxels almost the same as the first group of voxels benchmark, but it covers less of the map than model V1. Even though neither of the machine learning models beat the benchmark on pure labeling accuracy or map coverage they provide a better trade-off. While they are second best in the their respective metrics only by a fraction of a percent, which, in the case of the labeling accuracy for model V6, could be marked down to a statistical error, they perform significantly better in the other metric than their heuristics-based counterpart. This favorable trade-off is also evidenced by the raw accuracy, where both machine learning-based models beat all three of the benchmarks.

What is also interesting to mention is the consistency of each method across the different environments. While model V6 with MLP geometric features is the best option on average, it is also the least consistent one. This is especially visible on the *SeasideTown* environment and that example is explored more in depth in Section 4.1.5. On the other hand, the most consistent in its performance across all environments is the first voxel benchmark. Overall, while there are differences in the consistency of the methods and it is obvious some of the environments are more challenging than the others, it seems there is no clear evidence of one of them being harder for all of the methods than the others - different methods struggle with different issues.

Lastly, we can with certainty say that the lack of labeled voxels for certain methods is not due to insufficient coverage of the map with samples (images and poses), as the all voxels benchmark, which labels all encountered voxels, labels effectively 100 % of all maps.

4.1.5 Not seen semantic labels

Big limitation of the introduced machine learning methods is the fact that they work only for the fixed number of semantic labels they have been trained with. Out of the 52 labels present in TartanAir, only 43 have been shown to the models during training because not all environments contain all labels, and the environments have been split into the training ones and the testing ones (see Section 3.1). This forces the models to guess, if the not seen label should or shouldn't be placed in each voxel during inference.

Table 4.6: The average single view accuracies for the labels seen during training and not seen during training by the models.

	Model V1	Model V6 + MLP features
Seen labels	83.2 ± 8.1 %	87.8 ± 7.2 %
New labels	67.7 ± 20.2 %	62.5 ± 23.8 %

To see how the models react differently to labels they have seen and to those they have not seen during training, we calculated the average single view accuracy (see Section 4.1.2) for rays projected out from pixels with each of the 52 labels and averaged them by their presence (or lack thereof) in the training dataset. The results can be seen in Table 4.6.

As expected, the accuracies of the seen labels match the ones reported in Table 4.1 for the single view accuracy. The labeling of voxels with new labels is significantly less accurate but above 50 %, meaning our models are capable of better inference than if they only guessed. This is probably because some rules can be extrapolated for generic label assignment, for example, the label is more likely to be in the first few voxels along the ray than in the last ones.

However, both models V1 and V6 have very large standard deviation. For model V1, the best performing not seen label is *sea*, which the model was able to correctly place 94.0 % of the time and the worst performing label is *sandwich*, which the model was able to correctly place only 30.6 % of the time. For model V6 with MLP features, the best performing not seen label is *cloth* with 83.3 % accuracy and the worst performing one is interestingly *sea* with only 9.4 % accuracy.

While labels *sandwich* (accuracy on model V6 48.7 %) and *cloth* (accuracy on model V1 84.9 %) are not very remarkable from the point of view of the other model, the label *sea* is interesting, as it is the best performing one for model V1 and the worst performing one for model V6. Exploring how these two models react when presented with the *sea* label allows us to explore a crucial difference between the two models.

The label *sea* is present only in one environment - *Seasidetown* and in a very specific way. As can be seen in Figure 4.6, the sea is a flat, single voxel thick patch of voxels. Except along the shore, the voxels to which the *sea* label belongs will be the only ones a ray projected from a voxel with the same label encounters. Model V1 tends to label all voxels it can, the unique positioning of voxels with this label makes it hard to label them incorrectly, boosting model V1’s accuracy. On the other hand, model V6 seems to be a bit more conservative and avoids labeling the distinct, yet unknown semantic label and geometric context combination. It is interesting to note, that while *Seasidetown* has significantly worse raw accuracy and map coverage than the other environments for model V6 (see Tables 4.3 and 4.5), it’s labeled voxels accuracy is comparable to *Japanesealley* (see Table 4.4), suggesting the refusal to place the *sea* label is the main culprit behind the poor performance and with a more complete training dataset the model would perform better.

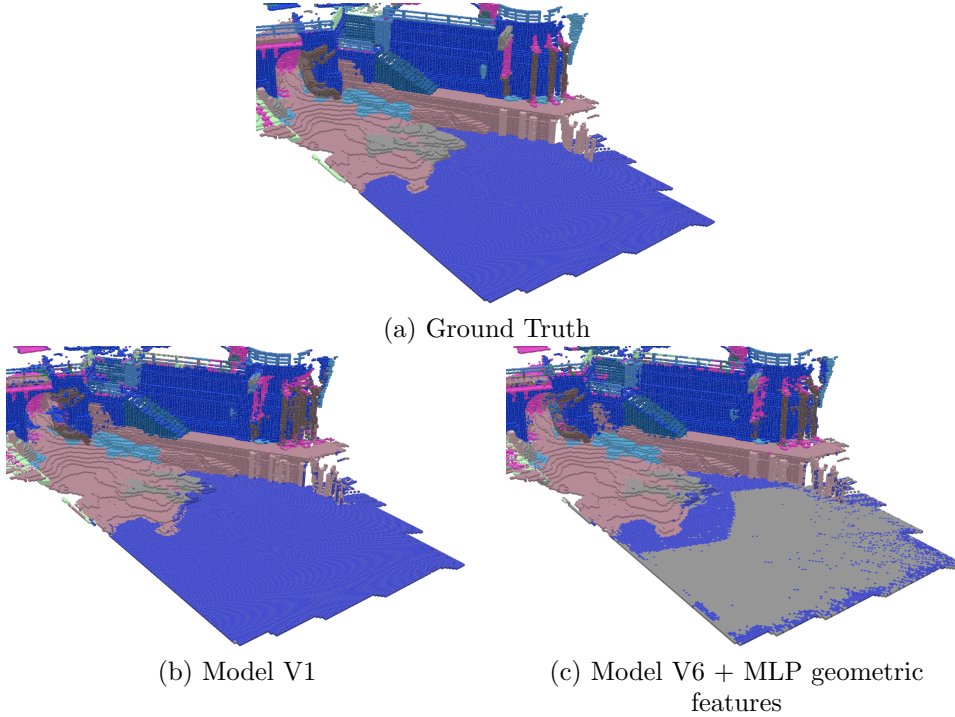


Figure 4.6: Part of the environment *Seasidetown* labeled by the two machine learning models compared to the ground truth. The *sea* label (light purple) is placed correctly by model V1 while being ignored (marked *unlabeled* - grey) by model V6.

4.2 Resource usage

4.2.1 Processing time

As we need G-SVOM to run real-time on the robot, we need it to be sufficiently fast - capable of running at least at 10 Hz, ideally 20 Hz. To verify that, we ran the whole voxel mapping pipeline on one of the trajectories (including the point cloud processing and combined map creation) and timed the two segments of merging semantic labels from an image to the map: data processing and model inference. The data processing step includes extracting the semantic labels from the images, filtering those marked as *unlabeled*, extracting the density vectors and geometric contexts (if needed), calculating the ray directions and then, once the inference

step is done, placing the labels into the voxels selected by the label association method. The inference step is the forward pass of the semantic label to voxel association method, including the processing of the geometric contexts into the geometric features (if needed).

Table 4.7: The time needed to merge semantic labels from a 640 by 480 pixel image into a voxel map.

	First	Group	All	V1	V6
Data processing	98 ms				218 ms
Inference	44 ms	45 ms	7 ms	13 ms	598 ms

It takes the Numba cuda library optimizer (the library allowing us to run things in parallel on the GPU) a few runs of each function to optimize it for the best performance. For that reason we average only the last 10 measurements. The experiment has been performed on a computer with Intel Core i7 7th generation processor and Nvidia GeForce GTX 1070 graphics card. The mapper parameters were the same as during the training and evaluation of the label association models presented in Table 3.2. The only other important parameter to mention is the size of the passed in images, which is 640 by 480 pixels. The measured processing times can be seen in Table 4.7.

Table 4.8: The time needed to merge semantic labels from a 640 by 480 pixel image into a voxel map, provided we sample only voxels from every fourth row and every fourth column.

	First	Group	All	V1	V6
Data processing	9 ms				19 ms
Inference	1 ms	17 ms	1 ms	1 ms	46 ms

Unfortunately, none of the proposed methods is fast enough to be capable of running real-time. Luckily, our methods work on per-pixel basis, so we can easily subsample the images, reducing the number of pixels we have to process, and therefore speeding up the semantic label merging. The densest we can sample the 640 by 480 images and still be able to run model V6 with MLP geometric features real-time is taking a pixel from every fourth row and every fourth column, reducing the number of sampled pixels sixteen times from 307 200 to 19 200 and therefore the number of pixel-voxel correspondences from 39 321 600 to just 2 457 600 (each pixel can label a ray of 128 voxels). This is a large reduction in the number of sampled pixels, but it has no effect on the labeled map accuracy (see Section 4.1.4). For the other four methods, sampling every second pixel was enough. The run times for the subsampled images can be seen in Table 4.8.

4.2.2 Memory requirements

While modern computers tend to have an abundance of random access memory (RAM), video random access memory (VRAM) still tends to be a somewhat scarce resource. G-SVOM makes heavy use of the GPU and therefore of VRAM. For that reason we need to be aware of how much VRAM is required.

G-VOM itself (the geometric backbone of G-SVOM) never needed more than 1 GB of memory during our experiments both in simulation and on real data. To more in depth explore how much memory each of our semantic label to voxel association methods use, we calculate the sizes of the arrays needed to run them.

For each pixel in an image, each model needs an input vector of length n_{in} , an output vector of length n_{out} and maybe some auxiliary helper vectors n_{aux} . Additionally,

each model has a certain number of weights/parameters n_{param} . By adding these together, we get the number of variables needed to process one 640 by 480 pixel image:

$$n_{run} = 640 \cdot 480 \cdot (n_{in} + n_{out} + n_{aux}) + n_{param}. \quad (4.2)$$

All of our models output the label assignment vector of length $n_{out} = N$.

The three benchmarks take in the densities along the ray vector of dimension ${}_b n_{in} = N$ and they need no auxiliary vectors. While the first voxel and first group of voxels benchmarks technically have a parameter - the density threshold, given it is only one variable it is negligible and we can say ${}_b n_{param} = 0$. By substituting these expressions into Equation 4.2 and then getting the values from Table 3.2, we get how many variables we need to run the benchmarks in our experiments:

$${}_b n_{run} = 640 \cdot 480 \cdot (N + N + 0) + 0 = 614\,400 \cdot N = 614\,400 \cdot 128 = 78\,643\,200. \quad (4.3)$$

In addition to the densities along the ray vector, model V1 also needs the semantic label, making its input length ${}_{v1} n_{in} = N + M$. It doesn't need any auxiliary vectors, but has parameters. The first linear layer has ${}_{v1} n_{in} \cdot {}_{v1} n_{in}$ parameters and the second one has ${}_{v1} n_{in} \cdot N$ parameters. By substituting these expressions into Equation 4.2 and then getting the values from Table 3.2, we get how many variables we need to run model V1 in our experiments:

$$\begin{aligned} {}_{v1} n_{run} &= 640 \cdot 480 \cdot ((N + M) + N + 0) + ((N + M) \cdot (N + M) + (N + M) \cdot N) \\ &= 307\,200 \cdot (180 + 128) + (180 \cdot 180 + 180 \cdot 128) \\ &= 94\,673\,040. \end{aligned} \quad (4.4)$$

Calculating the variables for model V6 with MLP features is the most complicated because we have to work with the geometric contexts. As an input we take the semantic label and the direction of the ray, but instead of the densities along the ray vector, we get a vector of geometric context features making our input vector ${}_{v6} n_{in} = M + 9 + N \cdot G$.

To extract the geometric features, we use the auxiliary helper variables. For each occupied voxel, we need to extract a K^3 voxels around it, for which we need to know where in the map it is - its 3D index. We don't know in advance how many occupied voxels will the rays encounter, so to conserve memory, we assume it will be at maximum 40 % of the voxels within the view of a single image (within the field of view of the camera and the range of the fixed-length rays) and preallocate a buffer of this length. To map which buffer slot stores the geometric context for each voxel encountered by the rays, we got inspired by G-VOM and created a lookup table. To summarize, for each pixel, the auxiliary vectors contain N lookup table values, $\lfloor 0.4N \rfloor \cdot K^3$ geometric contexts and $\lfloor 0.4N \rfloor \cdot 3$ voxel coordinates, making ${}_{v6} n_{aux} = N + \lfloor 0.4N \rfloor \cdot (K^3 + 3)$. The storage space needed for the geometric feature vectors is already accounted for in the inputs.

Lastly, we need to account for parameters of both the model and the geometric feature extractor. The feature extractor's first layer has $K^3 \cdot 256$ parameters and the second layer has $256 \cdot G$ parameters. For the model, the first layer has $(M + N \cdot G + 9) \cdot 1024$ parameters, the second one $1024 \cdot 512$ and the last one has $512 \cdot N$.

By substituting these expressions into Equation 4.2 and then getting the values from Table 3.2, we get how many variables we need to run model V6 with MLP geometric feature extractor in our experiments:

$$\begin{aligned}
{}_{v6}n_{run} &= 640 \cdot 480 \cdot ((M + N \cdot G + 9) + N + (N + \lfloor 0.4N \rfloor \cdot (K^3 + 3))) + \\
&\quad (K^3 \cdot 256 + 256 \cdot G) + ((M + N \cdot G + 9) \cdot 1\,024 + 524\,288 + 512 \cdot N) \\
&= 307\,200 \cdot (2\,109 + 128 + 37\,460) + 190\,720 + 2\,692\,096 \\
&= 12\,197\,801\,216.
\end{aligned} \tag{4.5}$$

The n_{run} values represent the number of variables needed to run each method, however, each variable is a 2 byte floating point number, so to get the amount of VRAM each method needs we need to multiply the amount of variables by two:

$$n_{vram} = 2n_{run}. \tag{4.6}$$

The memory needed to run the semantic label to voxels association methods can be seen in Table 4.9. It is important to note, that these are only lower bound estimates, as we didn't account for sizes of the input data, the voxel maps themselves and other variables created in the process of data preparation and postprocessing.

Table 4.9: The minimum amount of VRAM needed to run each of the methods to merge a 640 by 480 pixel image into a map with parameters from Table 3.2.

	First	Group	All	V1	V6
VRAM required	157 MB			189 MB	24.4 GB

Compared to model V6, model V1 and the benchmarks use negligible amounts of memory. On top of that, the fact that these numbers are only lower bound estimates is especially the case for model V6, as the lookup table and voxel indexes are not stored as 16 bit floating point numbers, but as 32 or even 64 bit integers (depending on the computer architecture) to work for array indexing.

It is interesting to note, that 47 % of the 24.4 GB of memory is taken up by the auxiliary variables needed for the extraction and processing of the geometric contexts. This may be partially because of an inefficiency in the geometric feature extraction process: it is not only possible, but even frequent, that multiple rays pass through the same voxel and the geometric context is extracted and feature computed for it multiple times, even though the result is the same. This may also have some negative effects on the processing time.

4.3 Running G-SVOM on real data

The three benchmarks and the two machine learning models have been developed, trained and tested on simulated data, however, robots operate in the real world, not in simulation. For that reason, to truly know how well our methods perform we need to try them on inputs from an actual robot - test how good our methods are at sim-to-real transfer. We took a sensor payload box designed for the ANYbotics ANYmal D legged robot and walked around with it in a park near the ETH Zurich Zentrum campus. We picked this spot because it contains many of the same objects as the TartanAir environments and has a variety of features (both semantic and geometric) one might encounter in both constructed and natural environments - for example trees with overhangs we have to walk under, bushes we can walk through, rocks, but also staircases, trash cans and parked bicycles. The path we walked was approximately 430 m long and took us approximately 7 minutes to complete. It included an ascent and a descent of multiple staircases, passing underneath natural and constructed overhangs, walking through a bush and doing several loops and u-turns.

To build the geometric voxel map, the box is equipped with a $360^\circ \times 59^\circ$ Livox Mid-360 LiDAR and to gather semantic information we have 3 Tier IV C1 HDR cameras with a 120° field of view. The cameras are placed on three different sides of the box to cover as much of its surroundings as possible. For localization, we used the inertial-visual SLAM pipeline running on a Zed 2i stereo camera, which is also on board the payload box. As the sensors were hand-held, we had no need of a full robotic pipeline with planners or locomotion controllers. An image of our test setup can be seen in Figure 4.7.



Figure 4.7: The setup of the real world experiment: A hand-held sensory payload box, with a Livox LiDAR and a Tier IV HDR camera (highlighted in red).

The settings of G-SVOM during the real-world test were identical to those we used in simulation (see Table 3.2) with one exception - for training and evaluation in simulation we used our modified version of the geometric backbone to build maps of the entire environment, while for this experiment we kept the G-VOM standard rolling window around the robot.

G-SVOM needs semantically segmented images as an input. To get those from regular RGB images from the HDR cameras we used SAN: Side adapter network for open-vocabulary semantic segmentation [33]. The two main design decisions behind using this semantic segmentation method was the open-vocabulary feature, allowing us to just pass in the 52 semantic labels present in TartanAir without needing any additional processing, and previous good experience with SAN in the team.

As we didn't have a robot, we didn't have to run the method online. We recorded ROS bags and replayed them offline with G-SVOM running on the same computer we used with the simulated data. We passed the point clouds directly to the point cloud processing function and the images from the HDR cameras were first undistorted and then segmented by SAN. The raw camera images are full HD (1920 by 1080 pixels) and our methods struggle to fit even smaller images into memory and process them in time (see Sections 4.2.1 and 4.2.2). For that reason, we scale down each image 5 times and adjust the intrinsic calibration parameters accordingly. These shrunken down images then get passed to the semantic label to voxels association function.

We unfortunately don't possess ground truth labels for the real world environment, so we will have to be content with a qualitative evaluation of each method's performance. In spaces with a few large objects, as seen in Figure 4.8, G-SVOM is capable of creating semantically labeled voxel maps representative of the environment. The observations made for each method on the simulated data hold.

While that is also the case for scenes with large number of smaller structures,

the semantic labeling is significantly worse. The following section shows a series of images (Figures 4.10 through 4.13) of such a “cluttered” scene (shown in Figure 4.9) labeled by each of the methods and discusses the issues with each one. Afterwards, we present some additional general observations about running G-SVOM on the recorded dataset.

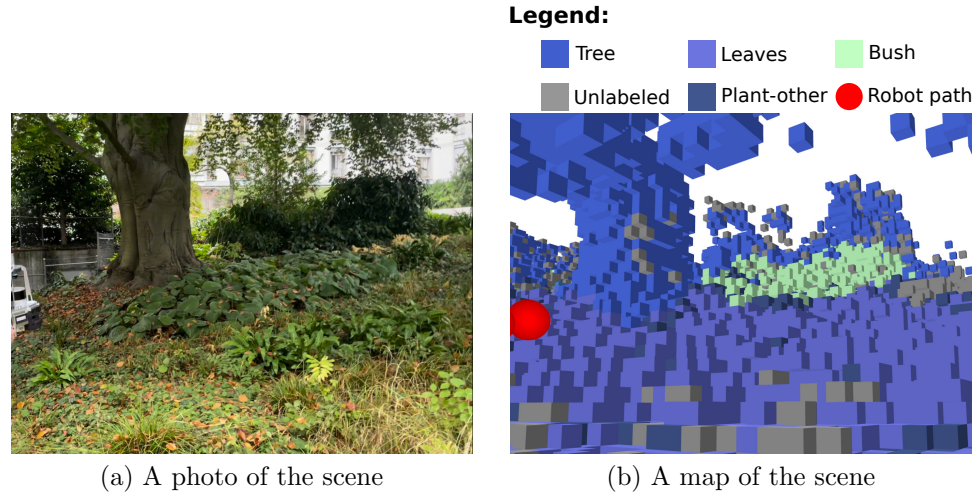


Figure 4.8: For spaces with a few large objects, G-SVOM is able to create semantically labeled voxel maps representative of the environment. The map in this image has been labeled by model V6.



Figure 4.9: A photo of the scene used to evaluate performance of the semantics to voxel association methods on real data. Its G-SVOM representations for each of the different methods can be seen in Figures 4.10 through 4.13.

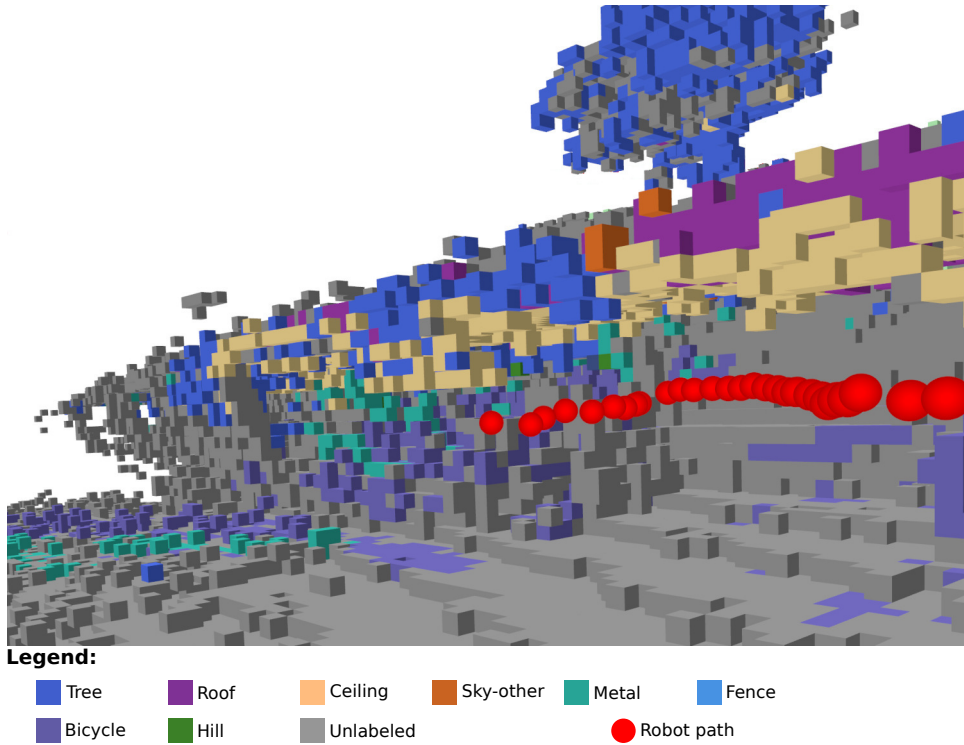


Figure 4.10: A snapshot of a map built from real world data labeled by the first voxel benchmark. Same as in the simulations, the first voxel benchmark suffers from label bleed around the edges and doesn't label most of the voxels because of occlusions. A lot of the ground is not labeled because of a failure of the semantic segmentation as can be seen in Figure 4.15.

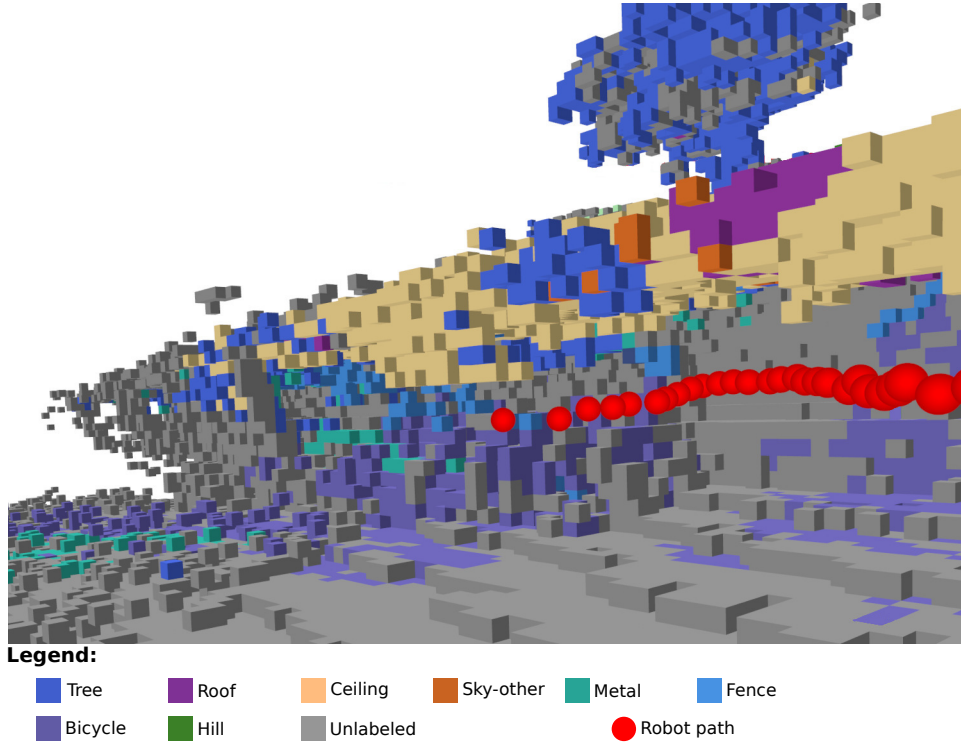


Figure 4.11: A snapshot of a map built from real world data labeled by the first group of voxels benchmark. This benchmark has similar label bleed issues as the first voxel benchmark. With regards to the occlusions, it starts showing a problem that is plaguing the rest of the methods - it labels more voxels along the ray than would be ideal, as can be seen with the *fence* label being placed into seemingly random voxels. This issue is so severe for the other methods, the first group of voxels benchmark may be the best performing method on the real data.

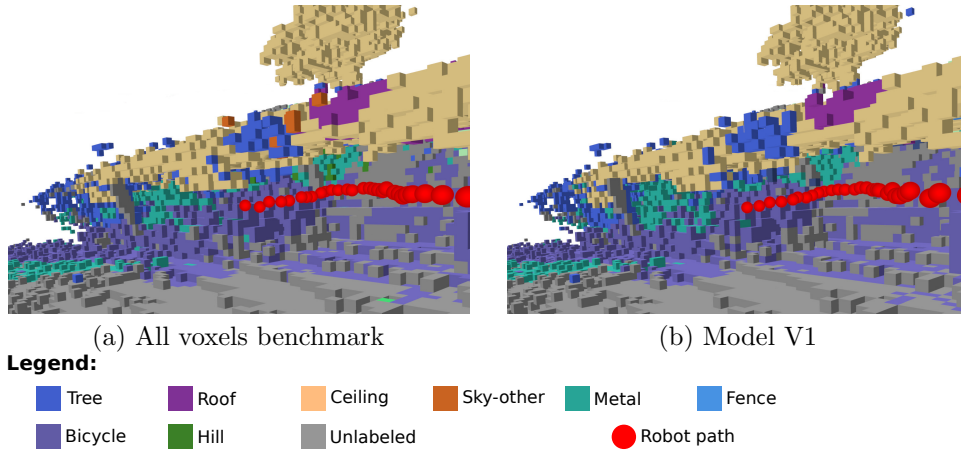


Figure 4.12: A snapshot of a map built from real world data labeled by the all voxels benchmark and model V1. Model V1 suffers the most from the problem mentioned with the first group of voxels benchmark (see Figure 4.11). It labels so many voxels along each ray it is indistinguishable from the all voxels benchmark (for which this isn't an issue, but a feature). As with the simulated data, model V1 is slightly more accurate with the labels it places, evidenced by the bleed of the *sky-other* label into the roof of the bicycle shed present for the benchmarks, but absent for model V1.

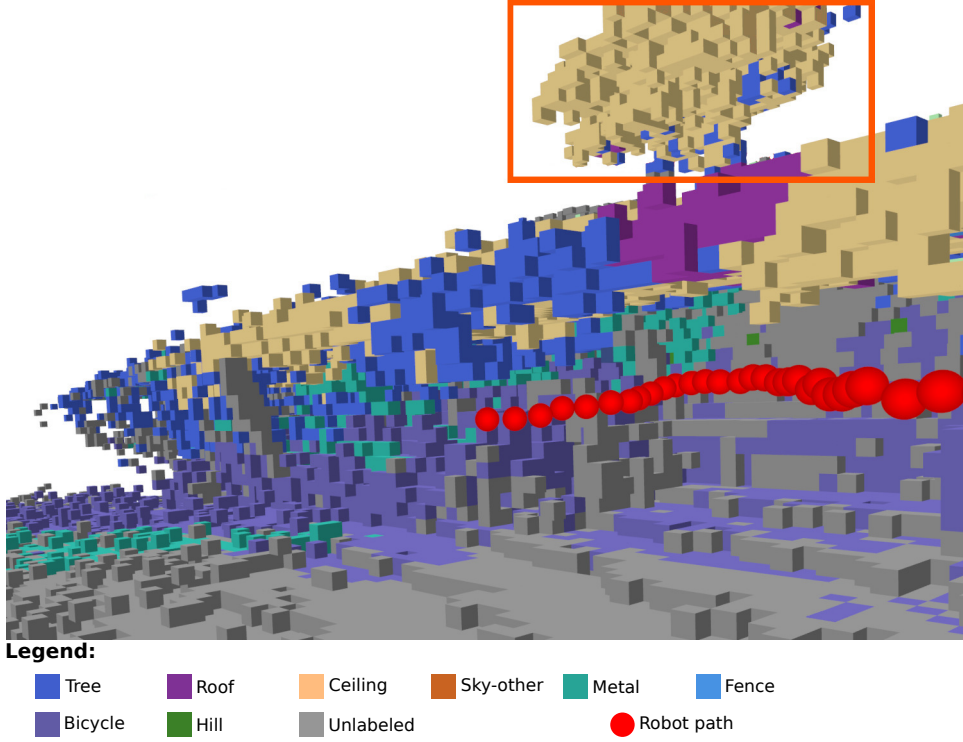


Figure 4.13: A snapshot of a map built from real world data labeled by model V6 with MLP geometric features. Model V6 is less aggressive with the labeling of voxels than model V1 (see Figure 4.12), however, it still labels too many of them as evidenced by the tree (marked with the orange rectangle), which is labeled as *ceiling*. In simulation model V6 was the best one at dealing with label bleed, but on the real data it seems to be comparable to model V1.

One feature we didn’t train or evaluate for were moving objects, which were present during the real world test in the form of humans walking around. Moving objects are a problem in general for mapping pipelines, and one of the motivations for adding semantics is their better classification and management. The simple moving object filter present in the geometric voxel mapping backbone of G-SVOM is insufficient at properly removing voxels that were once occupied by moving objects, causing them to leave trails in the map (shown in Figure 4.14). Because these voxels are in reality empty (they are falsely marked as occupied), they should not be marked with any label. This is not only a problem for accuracy, but might be an issue for further processing of the map, if we wanted to use the semantic labels to help filter out these falsely occupied voxels.

TartanAir contains perfectly segmented images. The real world RGB images segmented by SAN sometimes contain artifacts, which are shown in Figure 4.15. Around thin and see through objects (for example fences, bicycles and other mesh-like structures), the segmentation tends to be more of an approximation of the shape of the object rather than an pixel-perfect segmentation. This can exacerbate the label bleed issues around the edges of these narrow objects. Even though our set of 52 labels contains ones for *grass*, *wall* or *pavement*, large area of Figure 4.15 containing these vital features is marked as *unlabeled*. This explains why large parts of the map shown in Figures 4.10 to 4.13 do not have any labels. It might also explain the severe “overlabeling” of the voxels: If the unlabeled parts of the images were labeled, there could be many correctly labeled rays passing through these voxels,

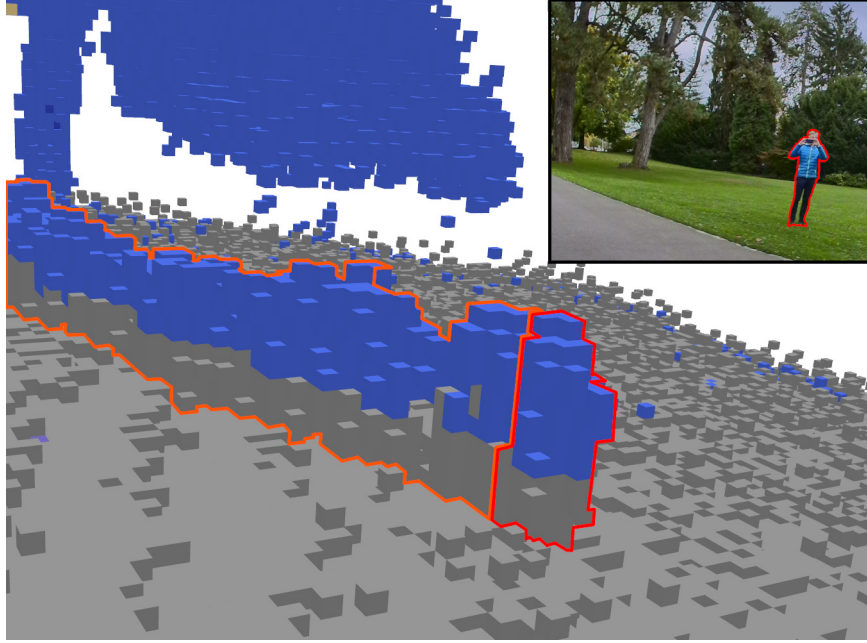


Figure 4.14: A walking human (highlighted in red) leaves a trail of falsely occupied voxels in the map (highlighted in orange), which are labeled as *tree* (blue) by the semantic label to voxel association method, in this example model V6 but this is an issue for all of them. As we might want to use semantics to help us remove these falsely occupied voxels, correct classification is crucial. The label *person* is not present in TartanAir and therefore wasn't available.

skewing the probability of the correct label being picked in their favor, but as that is not the case, the few erroneous rays are used instead to select the label.

This would suggest that SAN might not have been the best choice of semantic segmentation network for our purpose, however, one semantic-segmentation related issue is on us: label flickering. For example, if an object is called a *roof*, *ceiling* or a *floor* is solely dependent on the point of view of the observer. Additionally, is a leafy plant a *bush* or a *plant-other*? These uncertainties cause the semantic labels to change for certain objects between frames, which gets transferred to the map, causing objects to have voxels with different labels as the robot moves around them (see the roof above the robot in Figures 4.10 to 4.13). This could have been mitigated if we made sure to approach the TartanAir mesh number to COCO-Stuff label mapping process with this in mind.

The last issue to discuss with the real world experiment has to do with the odometry we used. The path we gathered the data on was a loop which started with the sensory payload box sitting on a bench and ended with the box being placed onto the exact same bench - the difference between the positions was less than a meter. However, the distance between the first and last robot positions as reported by our odometry was 13.35 m. This would mean an average drift of around 3 cm per meter walked, offsetting the map by one voxel every $\frac{0.15}{0.03} = 5$ m. This is significant, as our raytracing distance is approximately $128 \cdot 0.15 = 19.2$ m and the objects would drift while in our view. Additionally, we can observe this odometry drift having an effect on the map by stretching thin objects as seen in Figure 4.16.



Figure 4.15: An example of an image segmented by SAN. The metal support for the ceiling is missing significant sections and the grass, road and buildings are not detected at all.

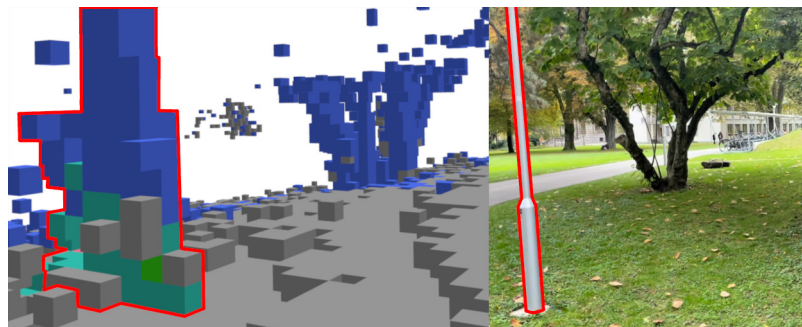


Figure 4.16: The thin lamp post (highlighted in red) has been spread into multiple voxels due to odometry drift. While its lower section is labeled correctly as *metal* (turquoise), its top is mislabeled as *tree* (blue).

4.4 Dependency on camera calibration

To project the rays from pixels into the voxel map we need to know the intrinsic calibration parameters of the camera and how it is positioned in the world (its extrinsic calibration parameters). While we trained and did the bulk of the evaluation of our methods on simulated data with perfect calibration, that won't be always the case in the real world as even a perfectly calibrated system can become slightly miscalibrated when brought to the test field, for example because of vibrations during transport and operation or due to temperature and humidity fluctuations. It is important to discuss, how our method might react to errors in both the intrinsic and extrinsic camera and LiDAR calibrations.

Our semantic label to voxel association methods are advertised as camera calibration-agnostic. That means that it is possible to take a method trained on a system with one camera and then use it on a system with a different one. However, if we use the wrong intrinsic camera matrix or assume we are in a different pose than we actually are, we will extract the wrong geometric contexts/density vectors and our methods won't give the correct results.

This is true for large errors, however, as the voxels are usually way bigger than the distance between the rays, small misalignments shouldn't cause the entire ray to be wrong. For example with the parameters of the TartanAir camera and our G-SVOM setup from Table 3.2, the furthest away voxel for each ray (approximately 20 m away from the camera) is still hit by 6 rays in the horizontal direction. Of course there might be errors in places where the ray encountered a voxel close to its edge, and that is where we believe our machine learning models should have an edge over the benchmarks, mainly model V6, as it has the geometric context to inform it, whether it should or shouldn't place the label into the "misaligned" voxel.

Chapter 5

Conclusion

5.1 Findings

In this thesis we present G-SVOM: A semantic voxel mapping pipeline using novel machine learning-based methods to associate semantic labels from images with voxels in the map. As the geometric backbone we use G-VOM [10], converting point clouds from LiDARs or depth cameras into a voxel map. It stores not only the occupancy of each voxel but also the voxel density, exact surface height and slope, allowing the robot to better reason about the environment. The most important of these heuristics is the idea of voxel density, representing the difference between a voxel being occupied because there is a blade of grass in it, and a voxel being occupied because it is part of a wall. There is one difference between the geometric representation of G-VOM and G-SVOM. Where G-VOM could only create a rolling map window around the robot, G-SVOM is capable of storing the map of the entire explored environment.

Our biggest contribution is the inclusion of a semantic label in each voxel. To leverage the research done into semantic segmentation of images, these labels are detected in an image from an RGB camera and then mapped to the voxels. G-SVOM includes 5 different methods of associating labels from an image with the voxels: three heuristics-based ones and two machine learning-based ones. We experimented with more machine learning-based methods, however, only the two, model version 1 and model version 6, have proven to be significant enough to use. All of the methods operate on a per-pixel basis, projecting a ray out and deciding into which voxels along the ray the pixel's label should be placed.

The two machine learning-based methods achieve the best accuracy, with model V1 labeling correctly on average 79.2 % of the map and model V2 labeling correctly on average 81.7 % of the map, beating the best performing heuristics based benchmark (label the first continuous group of occupied voxels along the ray) by 2.6 % and 5.1 % respectively. In addition to the increased accuracy of our models, their main benefit compared to most other machine learning models for semantics to geometry association is their agnosticism towards intrinsic camera parameters, allowing them to work on images from any camera without the need for retraining.

G-SVOM is implemented to take advantage of GPU parallelization as much as possible and can be run in real time.

The machine learning models were trained on a synthetic dataset. To see the performance of our semantic label to voxel association methods in the real world, we ran G-SVOM on a dataset collected from a sensoric payload box for the ANYbotics ANYmal D robot.

5.2 Limitations

There are two main issues encountered during the voxel labeling process: wrongly labeled voxels (“bleed”) and unlabeled voxels (occlusions). The machine learning-based methods offer a better overall accuracy, but when checking specifically for one of these two issues, there is always a heuristics-based benchmark better than the machine learning-based model. The first group of voxels benchmark has 0.5 % higher accuracy of labeled voxels (how many of the labeled voxels were labeled correctly) than model V6 and the all voxels benchmark labels 0.2 % of the map more than model V1. The machine learning-based models offer a better overall tradeoff, however, it would be nice if they also beat the benchmarks on the specific issues.

While the semantic label to voxel association methods can run real time, based on the size of the input images, they may have to be subsampled. Additionally, model V6 requires a lot of VRAM, which is an issue that can also be solved by subsampling the input images.

When running G-SVOM on the real-world dataset, the limitations of the synthetic training dataset show. It not only lacks certain important labels (for example “human”), but it is also relatively small (only 7 unique environments), making it hard for the models to generalize to completely new scenes. While spaces in the real world dataset with a few large objects are labeled well, the labeling performance could be better for more cluttered scenes. Quantitatively, this can be seen in the relatively high standard deviation of the accuracies of model V6 on the synthetic dataset compared to the rest of the methods.

5.3 Possible future work

Tuning the model architectures

While we explored the possibilities for different model architectures for a model based on the densities along the ray vector, we did not have the time to do such thing for the model using the full geometric contexts. As the semantic label to voxels along a ray assignment problem can be described as assigning a semantic feature to a voxel sequence, we believe attention-based architectures similar to the transformer described by [25] might be especially promising.

The same thing applies to the geometric feature extractors. We suspect that the reason for the poor performance of model V6 with the convolution-based feature extractor compared to its performance with the MLP feature extractor is the convolution-based feature extractor’s exceptionally poor design because convolutional neural networks are the standard for semantic segmentation. Maybe a network inspired by an architecture specially designed for the semantic segmentation of voxel maps without any semantic context from images, like for example [34], would perform better. Additionally, it might be interesting to include the semantic labels currently present in the voxels in the geometric feature extraction process.

Decreasing the resource requirements

What could also be improved is the image-to-voxel map label processing time and the amount of VRAM required by model V6. This could be achieved by not extracting the geometric contexts and calculating the geometric features when we require them, but either when combining the intermediate maps into the combined map, or on an independent periodic timer. When we would want to merge labels from an image into the map, we would only need to query the geometric features of the occupied voxels we encounter. This would not only decouple the geometric feature

extraction from the semantic label matching process, but it would also address the inefficiency of the geometric feature extraction being done multiple times for one voxel if multiple rays are passing through it.

While the benchmarks and model V1 also somewhat suffer from large processing times and would benefit from some optimization, the process to do so is less obvious.

Improve model generalization

What limits the ability of the machine learning-based models to generalize was the training dataset only consisting of 7 environments. Training the models on a bigger dataset, or perhaps a dataset also including some real world data should help them generalize better.

What might also help generalization would be including more input data. We motivated the geometric contexts for model V6 by mentioning how a human would solve the semantic label to voxel association task, however, we told only half the story. Humans don't only use the geometric context from the map and the label from the pixel, they also incorporate the semantic context of the pixel from the image. It would be interesting to explore, what would happen if we provided this context, for example, in the form of DINO features [35], as an input to the semantic label to voxel association method.

Improve the mapping pipeline

The methods introduced in this thesis only map a fixed label from the image to the map. Many state-of-the-art semantic mapping methods work with label probability distributions and sophisticated data merging methods allowing us to better model the labeling uncertainty. That is important for example for dealing with the label bleed issue and would help with further increasing the accuracy and interpretability of the final map.

Lastly, even though G-SVOM's maps contain the semantic labels, these labels are not used for anything. What they are mainly needed for is traversability estimation, for example for path detection. It would be interesting to not only include that, but then use these new traversability estimates to plan paths for a real robot (not only a sensoric payload box) to see how it would compare to state of the art robotic pipelines. Additionally, these semantic labels could be used for other things, like helping the mapper deal with moving objects.

Bibliography

- [1] J. Philion and S. Fidler, “Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d,” in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16*. Springer, 2020, pp. 194–210.
- [2] X. Meng, N. Hatch, A. Lambert, A. Li, N. Wagener, M. Schmittle, J. Lee, W. Yuan, Z. Chen, S. Deng *et al.*, “Terrainnet: Visual modeling of complex terrain for high-speed, off-road navigation,” *arXiv preprint arXiv:2303.15771*, 2023.
- [3] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. L. Rus, and S. Han, “Bev-fusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation,” in *2023 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2023, pp. 2774–2781.
- [4] J. Frey, S. Khattak, M. Patel, D. Atha, J. Nubert, C. Padgett, M. Hutter, and P. Spieler, “Roadrunner–learning traversability estimation for autonomous off-road driving,” *arXiv preprint arXiv:2402.19341*, 2024.
- [5] S. Thrun, M. Montemerlo, H. Dahlkamp, D. Stavens, A. Aron, J. Diebel, P. Fong, J. Gale, M. Halpenny, G. Hoffmann *et al.*, “Stanley: The robot that won the darpa grand challenge,” *Journal of field Robotics*, vol. 23, no. 9, pp. 661–692, 2006.
- [6] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [7] D. V. Lu, D. Hershberger, and W. D. Smart, “Layered costmaps for context-sensitive navigation,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 709–715.
- [8] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, “Elevation mapping for locomotion and navigation using gpu,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 2273–2280.
- [9] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “Pointpillars: Fast encoders for object detection from point clouds,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 12 697–12 705.
- [10] T. Overbye and S. Saripalli, “G-vom: A gpu accelerated voxel off-road mapping system,” in *2022 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2022, pp. 1480–1486.

- [11] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, “nvblox: Gpu-accelerated incremental signed distance field mapping,” 2024.
- [12] M. G. Besselmann, L. Puck, L. Steffen, A. Roennau, and R. Dillmann, “Vdb-mapping: A high resolution and real-time capable 3d mapping framework for versatile mobile robots,” in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, 2021, pp. 448–454.
- [13] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, 2013, software available at <https://octomap.github.io>.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [15] B. Agro, Q. Sykora, S. Casas, T. Gilles, and R. Urtasun, “Uno: Unsupervised occupancy fields for perception and forecasting,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 14 487–14 496.
- [16] J. F. Henriques and A. Vedaldi, “Mapnet: An allocentric spatial memory for mapping environments,” in *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8476–8484.
- [17] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering.” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [18] W. Zeng, W. Luo, S. Suo, A. Sadat, B. Yang, S. Casas, and R. Urtasun, “End-to-end interpretable neural motion planner,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8660–8669.
- [19] G. Erni, J. Frey, T. Miki, M. Mattamala, and M. Hutter, “Mem: Multi-modal elevation mapping for robotics and learning,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 11 011–11 018.
- [20] D. Maturana, P.-W. Chou, M. Uenoyama, and S. Scherer, “Real-time semantic mapping for autonomous off-road navigation,” in *Field and Service Robotics: Results of the 11th International Conference*. Springer, 2018, pp. 335–350.
- [21] P. Ewen, A. Li, Y. Chen, S. Hong, and R. Vasudevan, “These maps are made for walking: Real-time terrain property estimation for mobile robots,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7083–7090, 2022.
- [22] M. Grinvald, F. Furrer, T. Novkovic, J. J. Chung, C. Cadena, R. Siegwart, and J. Nieto, “Volumetric instance-aware semantic mapping and 3d object discovery,” *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 3037–3044, 2019.
- [23] L. Gan, R. Zhang, J. W. Grizzle, R. M. Eustice, and M. Ghaffari, “Bayesian spatial kernel smoothing for scalable dense semantic mapping,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 790–797, 2020.

- [24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.
- [25] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [26] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng *et al.*, “Ros: an open-source robot operating system,” in *ICRA workshop on open source software*, vol. 3, no. 3.2. Kobe, Japan, 2009, p. 5.
- [27] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.
- [28] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A llvm-based python jit compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6.
- [29] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [30] W. Wang, D. Zhu, X. Wang, Y. Hu, Y. Qiu, C. Wang, Y. Hu, A. Kapoor, and S. Scherer, “Tartanair: A dataset to push the limits of visual slam,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 4909–4916.
- [31] H. Caesar, J. Uijlings, and V. Ferrari, “Coco-stuff: Thing and stuff classes in context,” in *Computer vision and pattern recognition (CVPR), 2018 IEEE conference on*. IEEE, 2018.
- [32] D. P. Kingma, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [33] M. Xu, Z. Zhang, F. Wei, H. Hu, and X. Bai, “San: Side adapter network for open-vocabulary semantic segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [34] S.-S. Huang, Z.-Y. Ma, T.-J. Mu, H. Fu, and S.-M. Hu, “Supervoxel convolution for online 3d semantic segmentation,” *ACM Transactions on Graphics (TOG)*, vol. 40, no. 3, pp. 1–15, 2021.
- [35] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 9650–9660.